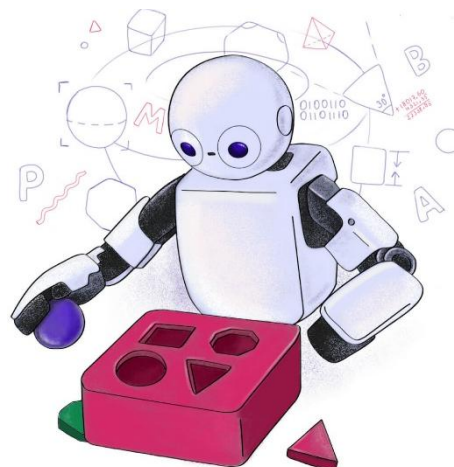
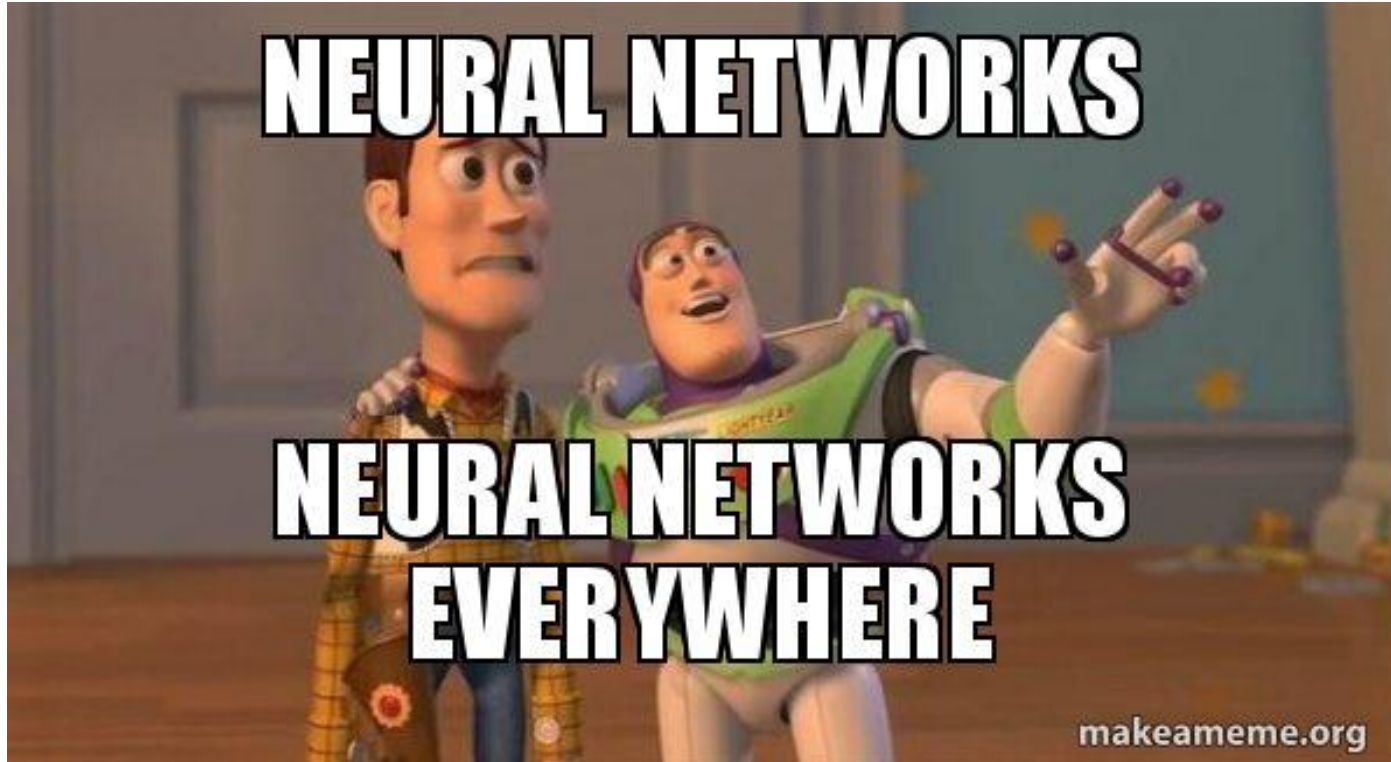


C24 – Inteligência Artificial: *Redes Multilayer Perceptron*



Inatel

Felipe A. P. de Figueiredo
felipe.figueiredo@Inatel.br



Redes neurais são os modelos de ML mais usados atualmente.

ChatGPT

Claude

Gemini

FaceID

Filtros do Instagram

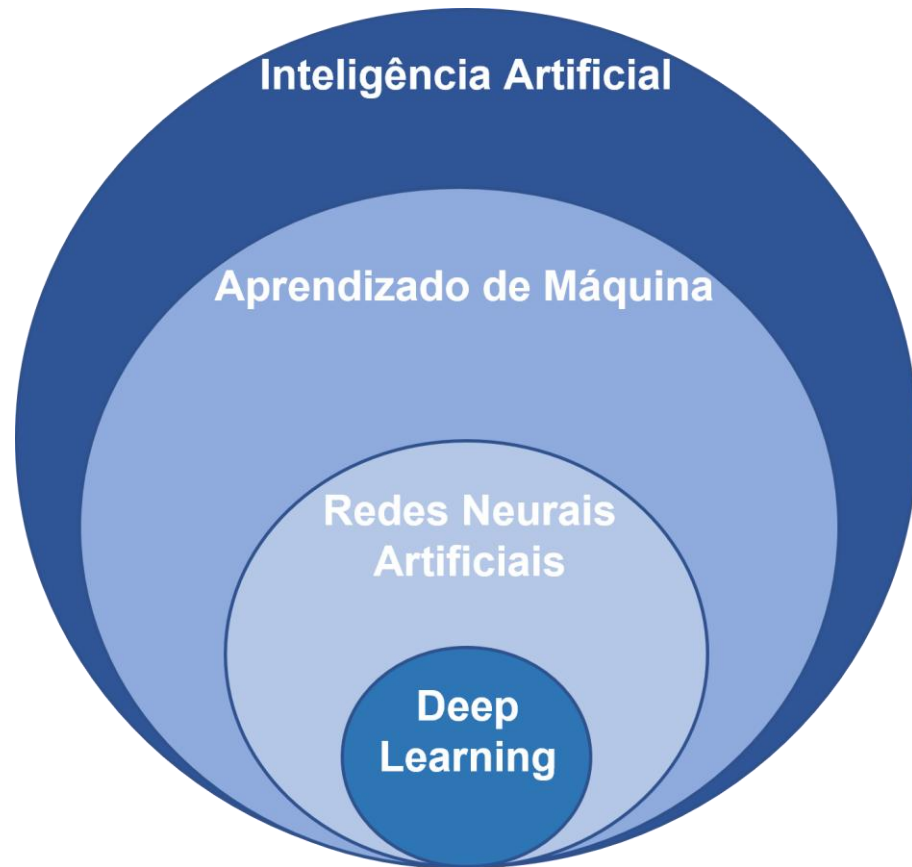
Alexa

Recomendação (Netflix, Spotify)

Google tradutor

Filtros de spam

Redes neurais artificiais



Subárea do aprendizado de máquina.

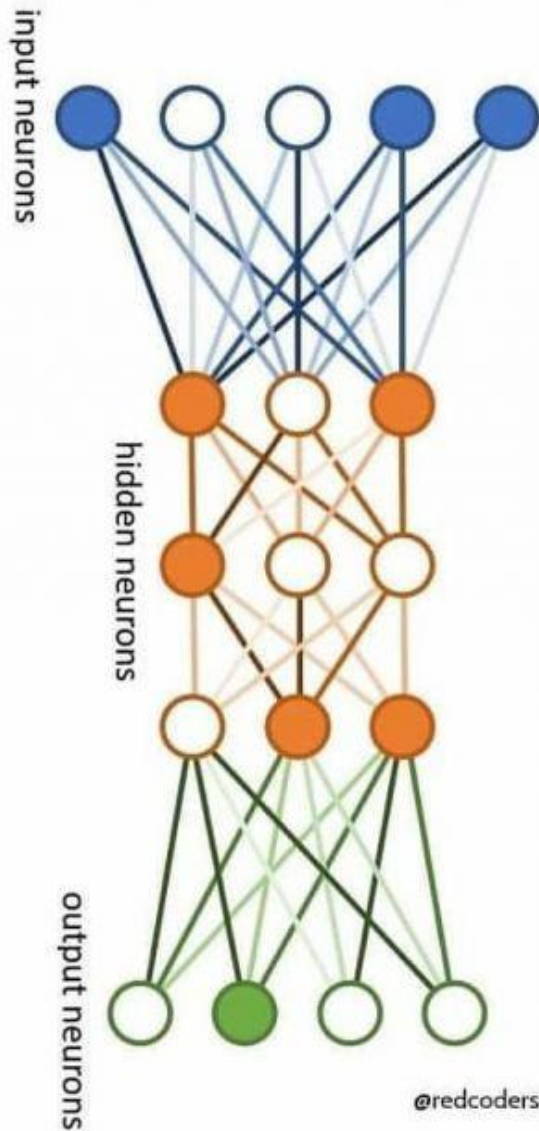
Ela engloba o aprendizado profundo (*deep learning*).

O que é uma rede neural
artificial?

**THIS IS A NEURAL
NETWORK.**

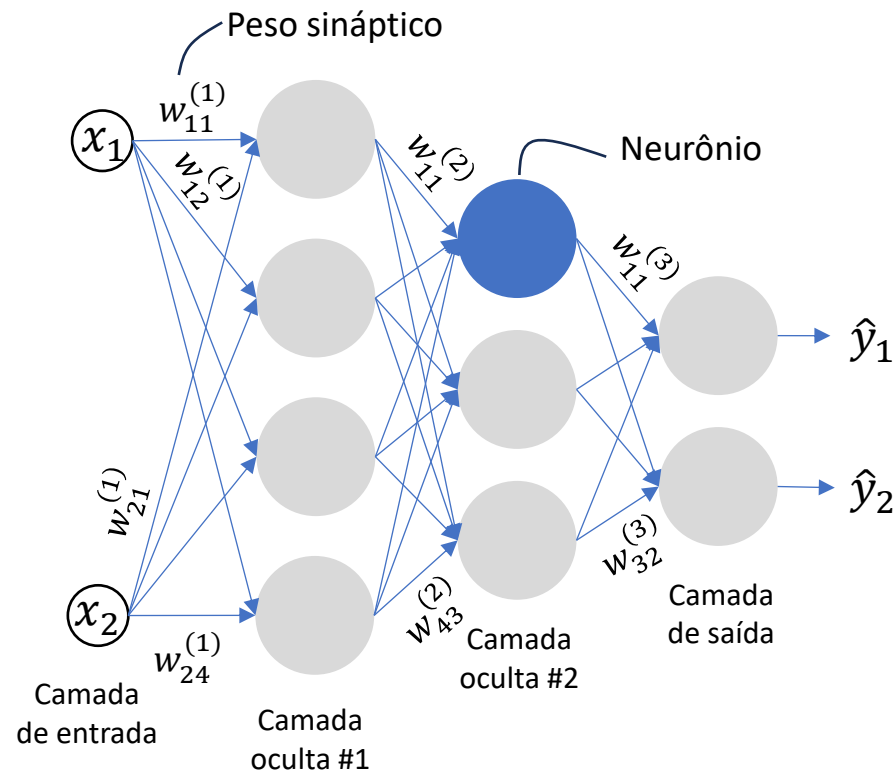
**IT MAKES MISTAKES.
IT LEARNS FROM THEM.**

**BE LIKE A NEURAL
NETWORK.**



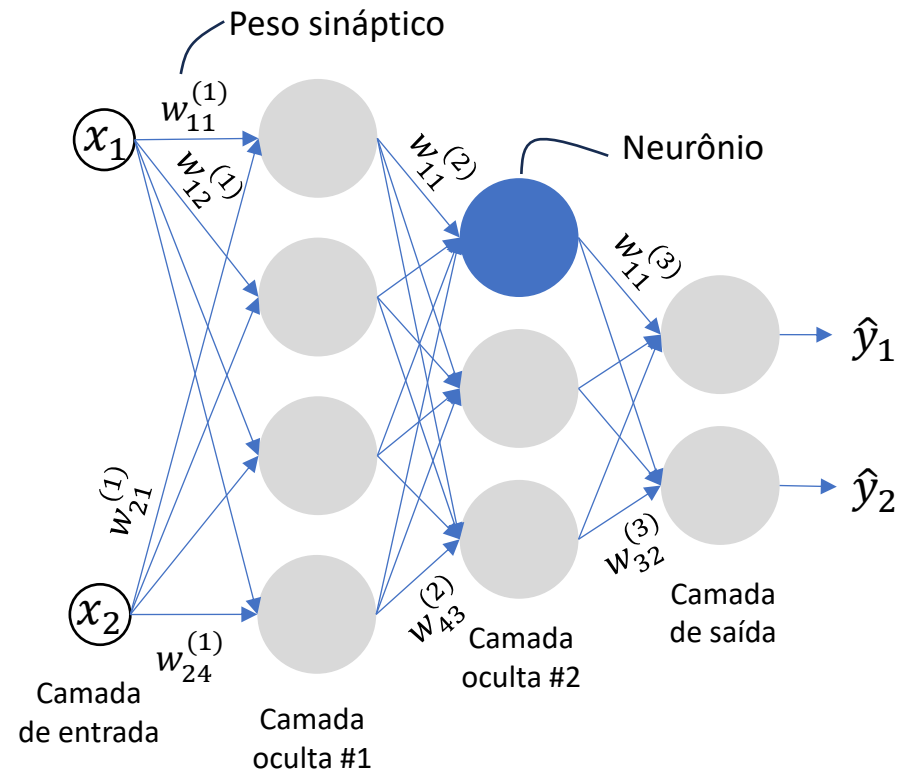
Modelo com múltiplas
entradas, com uma ou mais
camadas, onde cada uma
delas contém vários
neurônios interconectados
que geram uma ou mais
saídas.

O que é uma rede neural artificial?



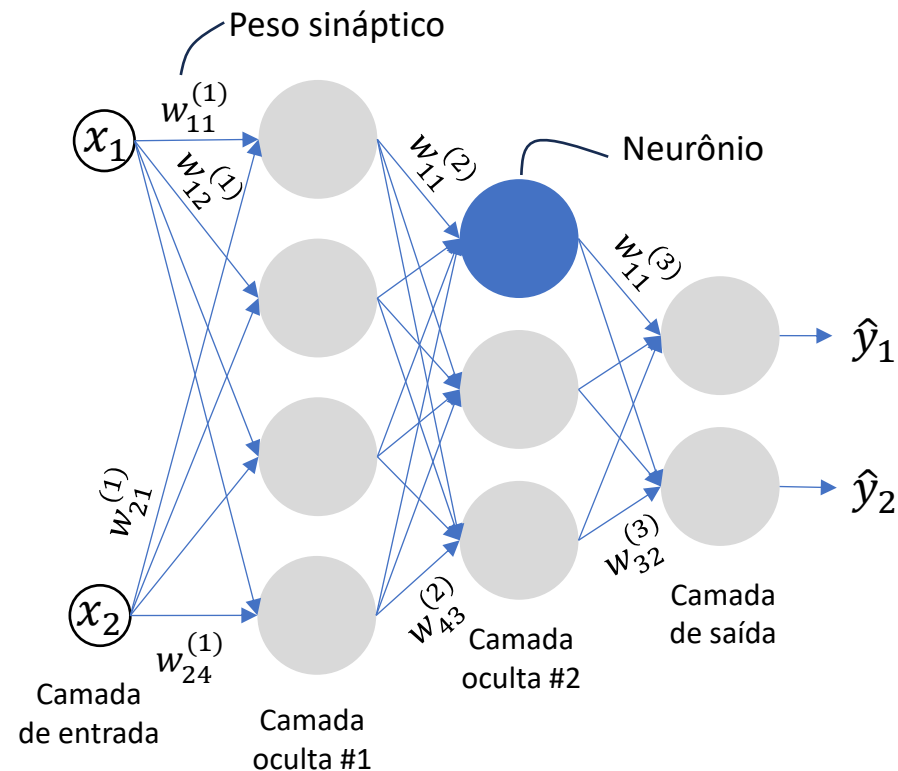
- **Redes neurais artificiais** são modelos matemáticos **inspirados** pelo funcionamento do cérebro dos animais.
- Elas são capazes de realizar **tarefas de aprendizado de máquina** com grande versatilidade e eficácia.
 - regressão,
 - classificação,
 - clusterização,
 - detecção e segmentação de objetos,
 - rastreamento de objetos,
 - etc.

O que é uma rede neural artificial?



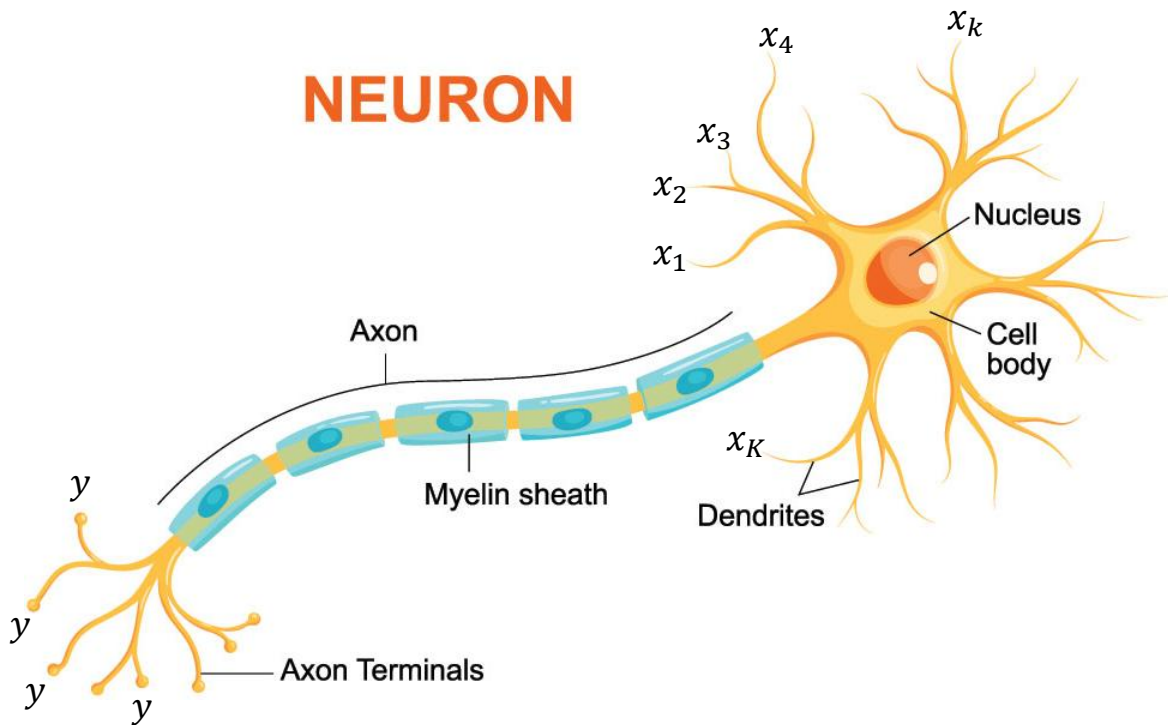
- RNAs são representadas como
 - sistemas de **neurônios (ou nós) interconectados**,
 - organizados em **camadas**,
 - que **aprendem, a partir de dados**, uma **função que relaciona as entradas às saídas**.

O que é uma rede neural artificial?



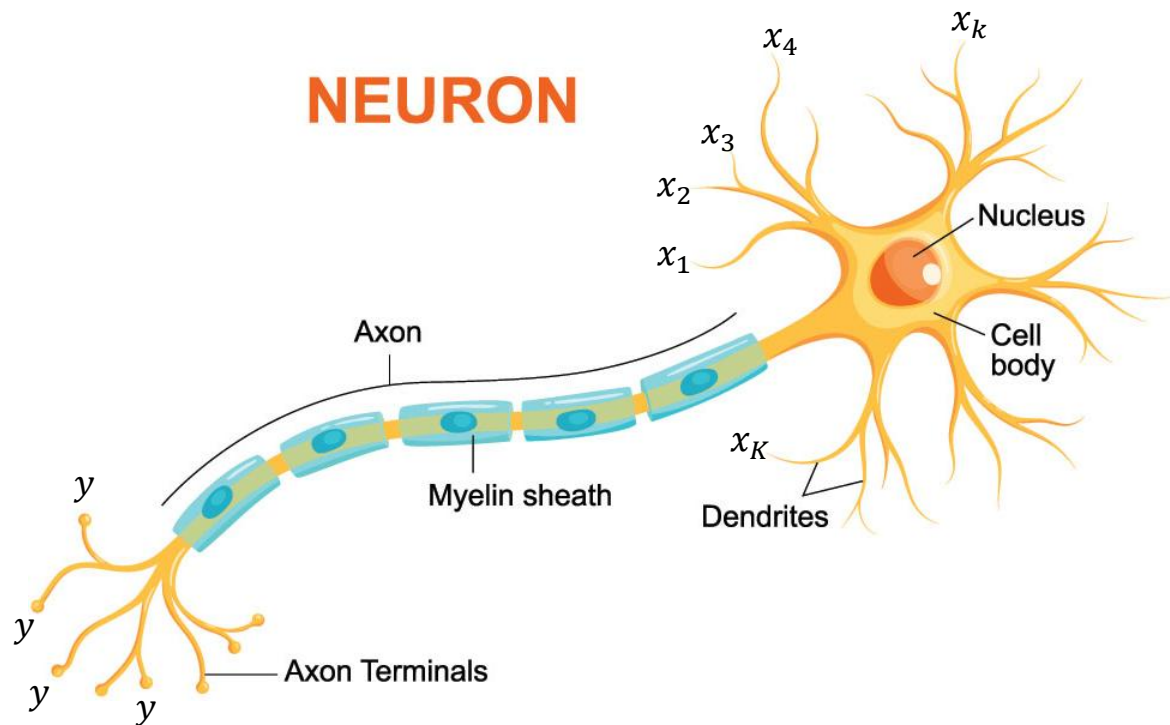
- Antes de entendermos como funciona uma rede neural, **precisamos primeiro discutir o princípio básico de funcionamento um neurônio biológico** e como ele é usado para modelar um **neurônio artificial**.

Neurônio biológico



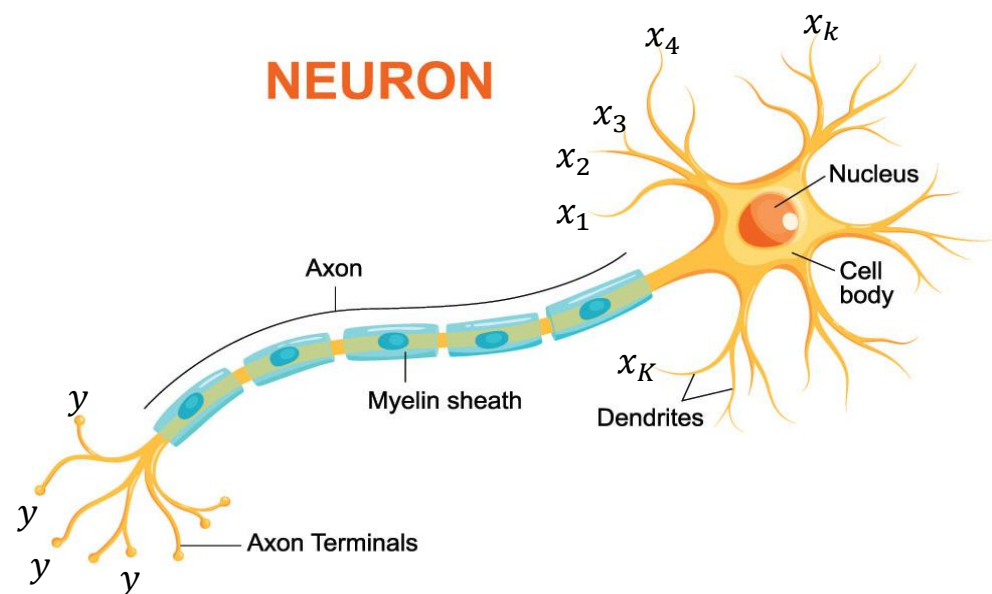
- São **células** que possuem **mecanismos eletroquímicos** para realizar a **transmissão de sinais elétricos** (i.e., informações) **ao longo do sistema nervoso**.
- Eles têm três partes fundamentais:
 - os **dendritos**,
 - o **corpo celular**, também chamado de **soma**,
 - e o **axônio** com seus terminais.

Neurônio biológico



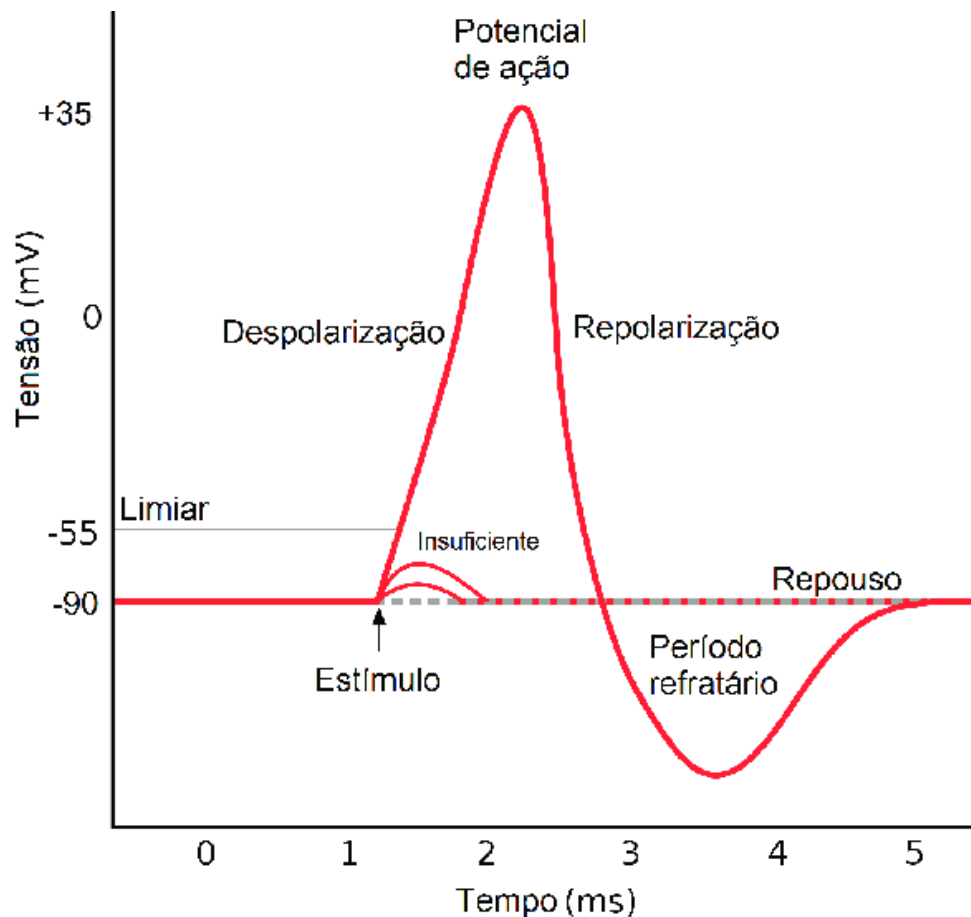
- Os **dendritos** recebem **estímulos** vindos de outros neurônios e os levam em direção ao **corpo celular**.
- O **corpo celular** realiza a **integração dos estímulos** e gera um impulso caso um **limiar de ativação seja ultrapassado**.
- O **axônio** envia os **impulsos** a outros neurônios através de seus **terminais**.

Neurônio biológico



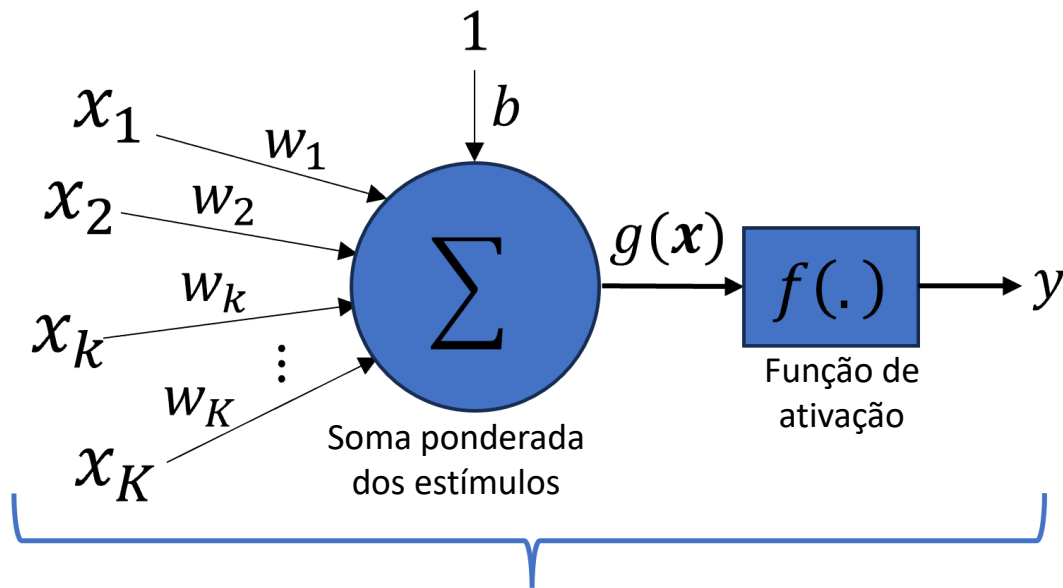
- Os neurônios se comunicam uns com os outros através das **sinapses**.
- **Sinapses** são os **pontos de contato** entre os **dendritos** de um neurônio e os **terminais do axônio** de outros neurônios.
- As sinapses podem ser:
 - **Excitatórias**: aumentam a chance de disparo.
 - **Inibitórias**: diminuem a chance de disparo.
 - **Fortes**: a informação de um neurônio é muito importante para o próximo.
 - **Fracas**: a informação de um neurônio não é tão importante para o próximo.

Neurônio biológico



- Em termos bem simples, o **neurônio** funciona da seguinte forma:
 - Ele **recebe estímulos elétricos** através dos dendritos.
 - Esses **estímulos são integrados** no corpo celular (soma).
 - Se a **integração dos estímulos** exceder um certo **limiar de ativação**, o **neurônio** gera um **impulso** (ou **potencial de ação**) que é enviado pelos terminais do axônio a outros neurônios através das **sinapses**.

Neurônio artificial

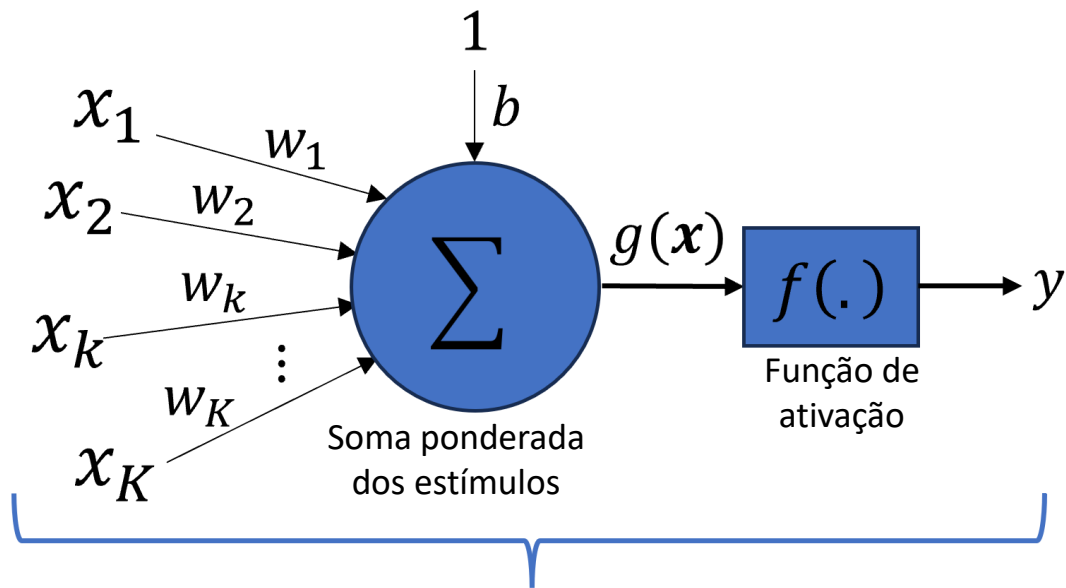


$$y = f(g(\mathbf{x})) = f\left(\left(\sum_{i=1}^K w_i x_i\right) + b\right)$$

- Baseado no entendimento do funcionamento do neurônio biológico, a partir de **meados da década de 40**, pesquisadores propuseram o **modelo matemático ao lado**.
 - É uma **simplificação e abstração** do funcionamento do neurônio biológico.

Qual modelo de ML aprendido anteriormente se assemelha ao neurônio artificial?

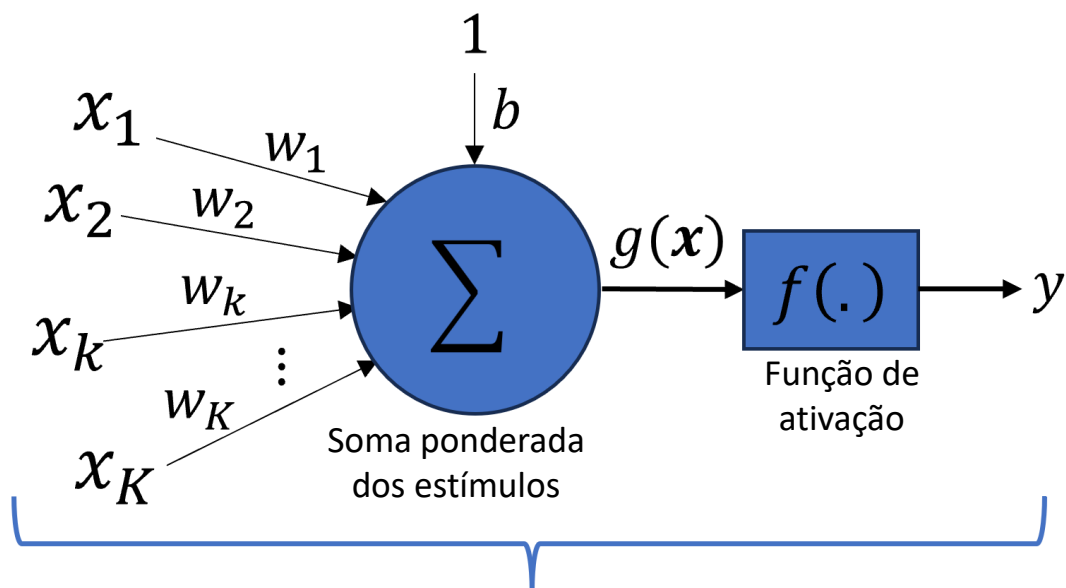
Neurônio artificial



$$y = f(g(\mathbf{x})) = f\left(\left(\sum_{i=1}^K w_i x_i\right) + b\right)$$

- O neurônio se assemelha ao **regressor logístico**.
- Os **estímulos**, $x_k, k = 1, \dots, K$, são **multiplicados** pelos **pesos sinápticos**, $w_k, k = 1, \dots, K$, somados ao **peso de bias**, b , e o resultado, $g(\mathbf{x})$, é passado através de uma **função de ativação**, $f(\cdot)$.

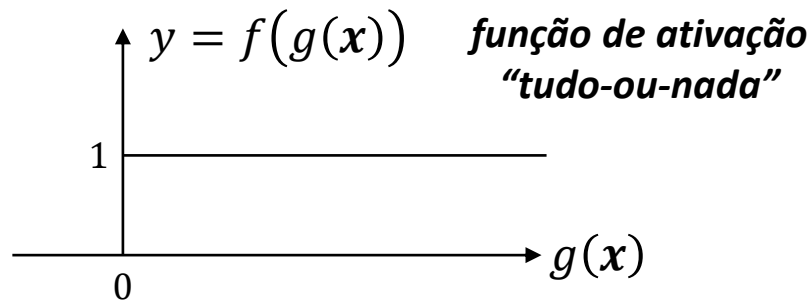
Neurônio artificial



$$y = f(g(\mathbf{x})) = f\left(\left(\sum_{i=1}^K w_i x_i\right) + b\right)$$

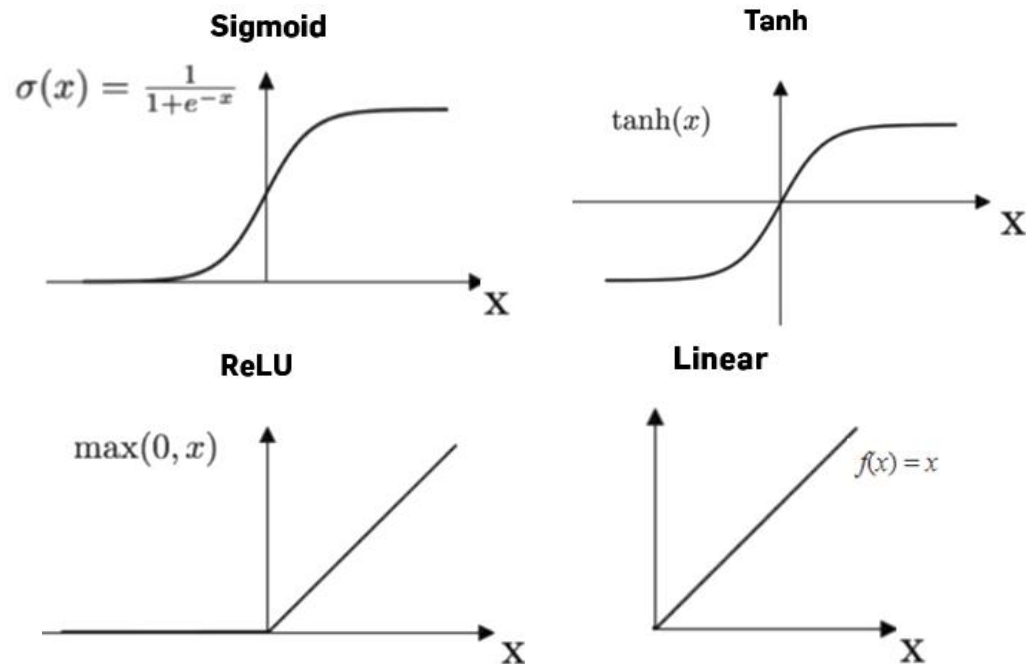
- No contexto de redes neurais os estímulos, $x_k, k = 1, \dots, K$, são chamados de **atributos**.
- O **peso de bias**, b , permite **ajustar o limiar de ativação** (ou ponto de disparo) da função de ativação **do neurônio**.
- Os **pesos sinápticos** w_k dão a **força de cada conexão** e se ela é **excitatória ou inibitória**.
- A média ponderada dos atributos, $g(\mathbf{x})$, é chamada de **ativação**.

Neurônio artificial



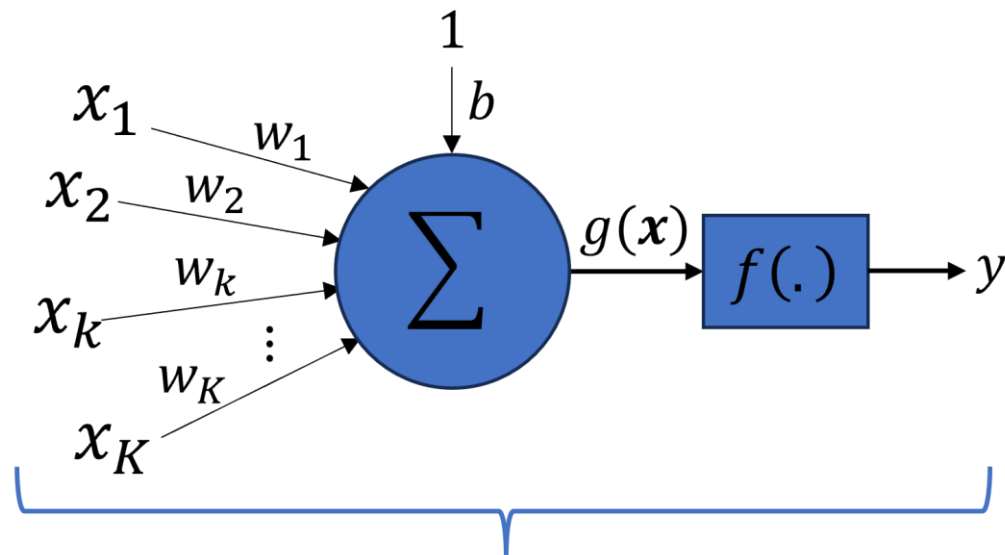
- No início dos estudos dos neurônios, a **função de ativação** era do tipo **tudo-ou-nada**, i.e., ativava ou não.
 - Em outras palavras, era um degrau unitário.
- Entretanto, ela não é mais usada porque sua **derivada é zero** para valores maiores e menores do que 0 e indeterminada em 0.
- Isso **impede o ajuste dos pesos** pelo algoritmo de **gradiente descendente**.

Neurônio artificial



- Assim, com o decorrer do tempo, funções de ativação **contínuas e deriváveis** foram propostas:
 - sigmoide (ou logística),
 - softmax,
 - tangente hiperbólica,
 - linear,
 - relu e suas variantes,
 - etc.
- **OBS.:** A função “linear” é usada, em geral, apenas na saída de redes neurais para regressão.

Neurônio artificial

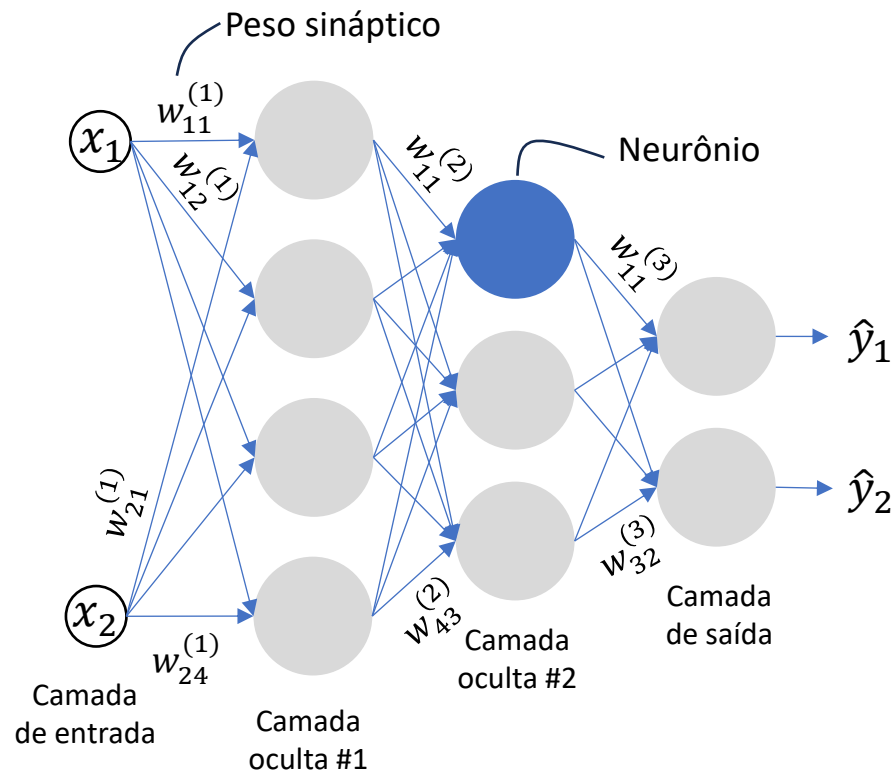


$$y = f \left(\left(\sum_{i=1}^K w_i x_i \right) + b \right)$$

Pesos ou
parâmetros

- Em geral, redes neurais artificiais são modelos de aprendizado supervisionado.
- Portanto, o **objetivo** do **treinamento** de uma rede neural é **encontrar os valores dos pesos** (bias e sinápticos) **de todos os neurônios** que **minimizem uma função de erro**.
- Por exemplo:
 - Classificação: entropia cruzada
 - Regressão: erro quadrático médio

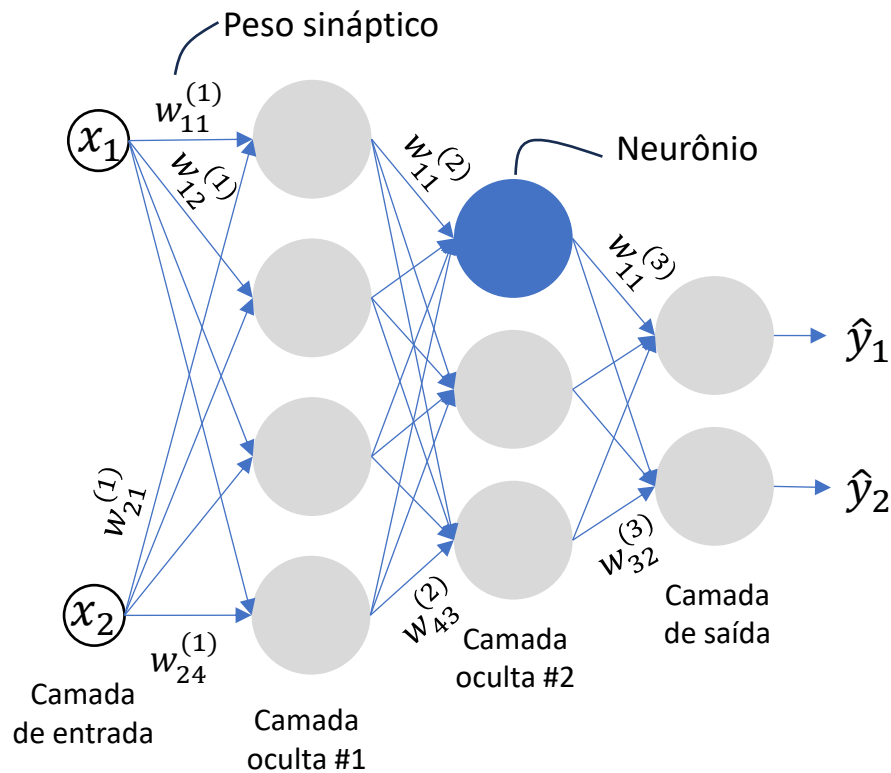
Neurônio artificial



- Os pesos são ajustados através de um **algoritmo de otimização iterativo** chamado de **retropropagação do erro**, o qual é baseado no gradiente descendente.

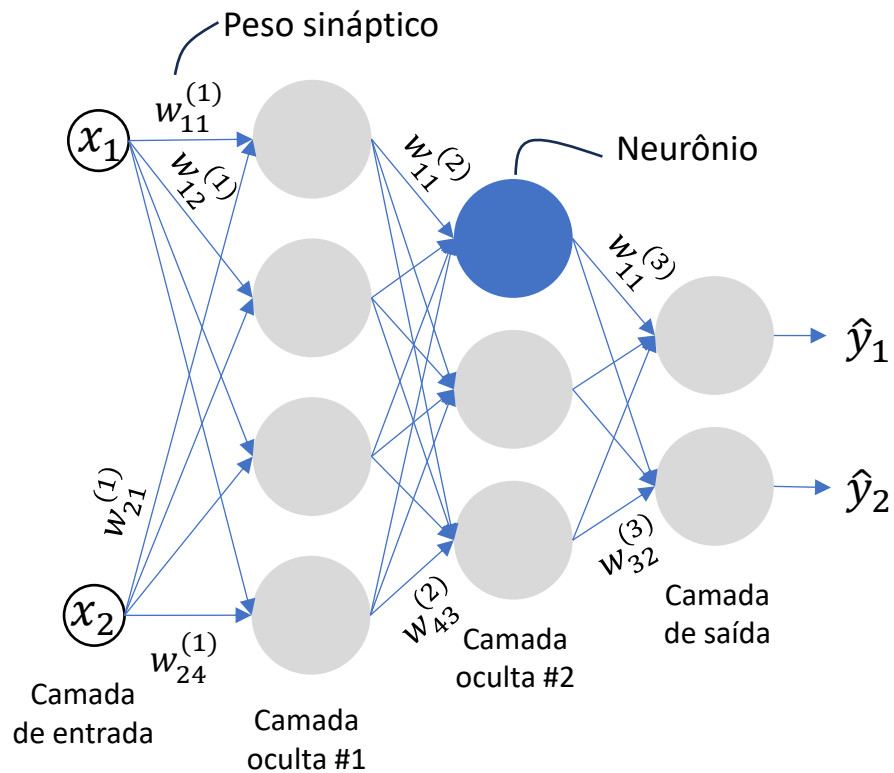
Redes neurais artificiais

Redes neurais artificiais



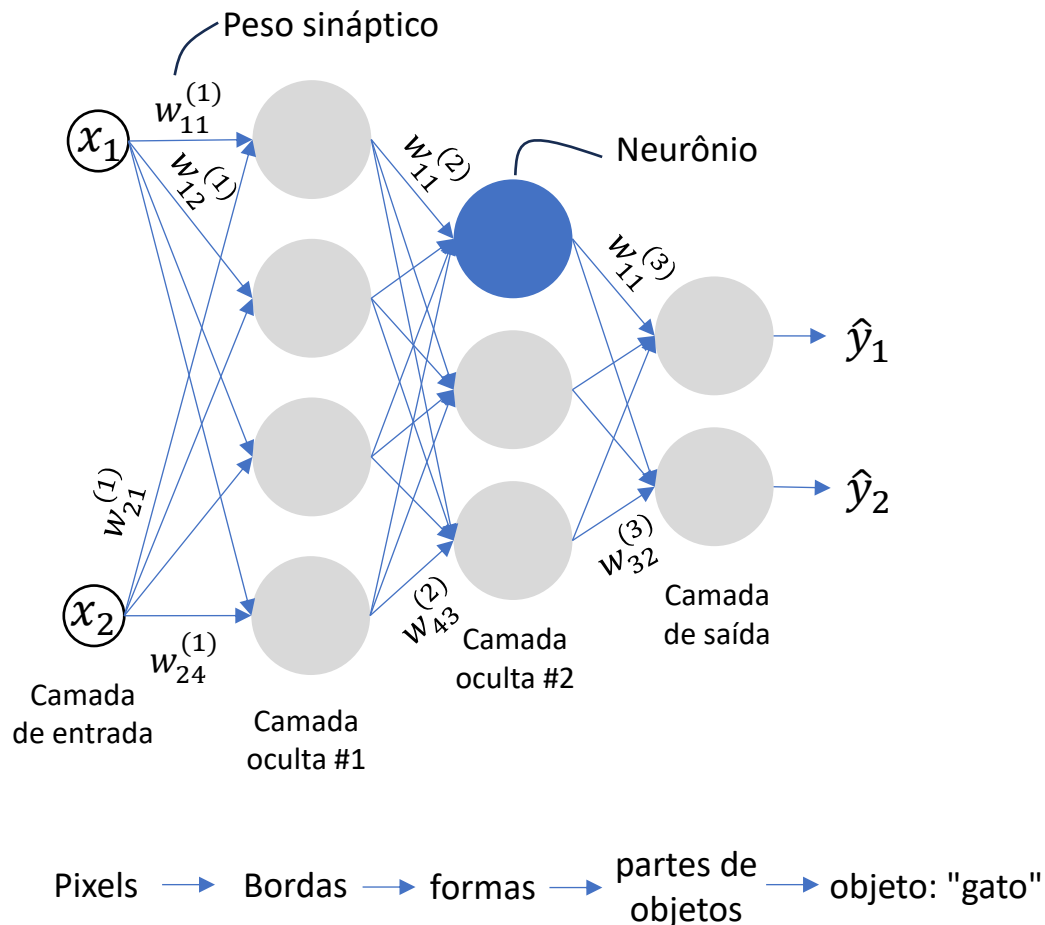
- As RNAs são formadas por uma camada de entrada, zero ou mais camadas intermediárias e uma camada de saída.
- A camada de entrada é o ponto de transferência dos atributos para dentro da rede.
 - Essa camada **não tem pesos**.
 - Ela pode ser usada para aplicar **transformações de dimensão** aos dados de entrada.
- Cada camada, exceto a de entrada, possui um ou mais neurônios.

Redes neurais artificiais



- As camadas intermediárias são também chamadas de **ocultas** ou **escondidas**.
- As camadas ocultas **transformam progressivamente** os **dados de entrada em representações internas que sejam úteis para produzir a saída desejada**.

Redes neurais artificiais

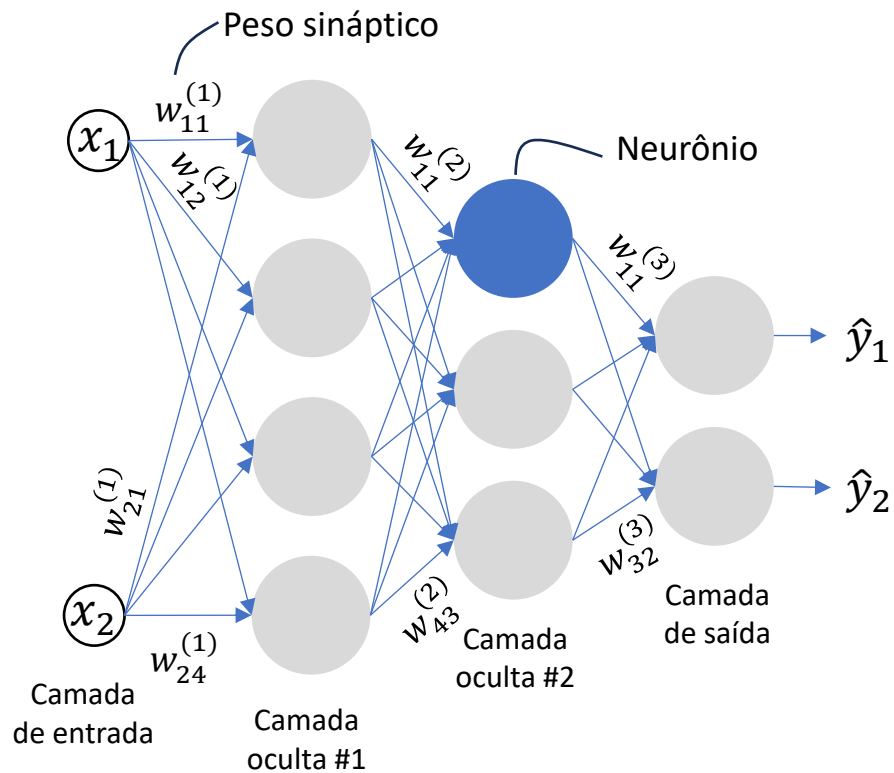


- A ideia é que as **representações** sejam construídas **hierarquicamente**:
 - camadas iniciais detectam padrões mais básicos,
 - enquanto camadas posteriores combinam esses padrões em estruturas progressivamente mais complexas.
- O primeiro tipo de rede neural proposto foi o **perceptron de múltiplas camadas** (do inglês, *Multilayer Perceptron* - MLP).
 - Rede proposta em 1958 por Frank Rosenblatt.



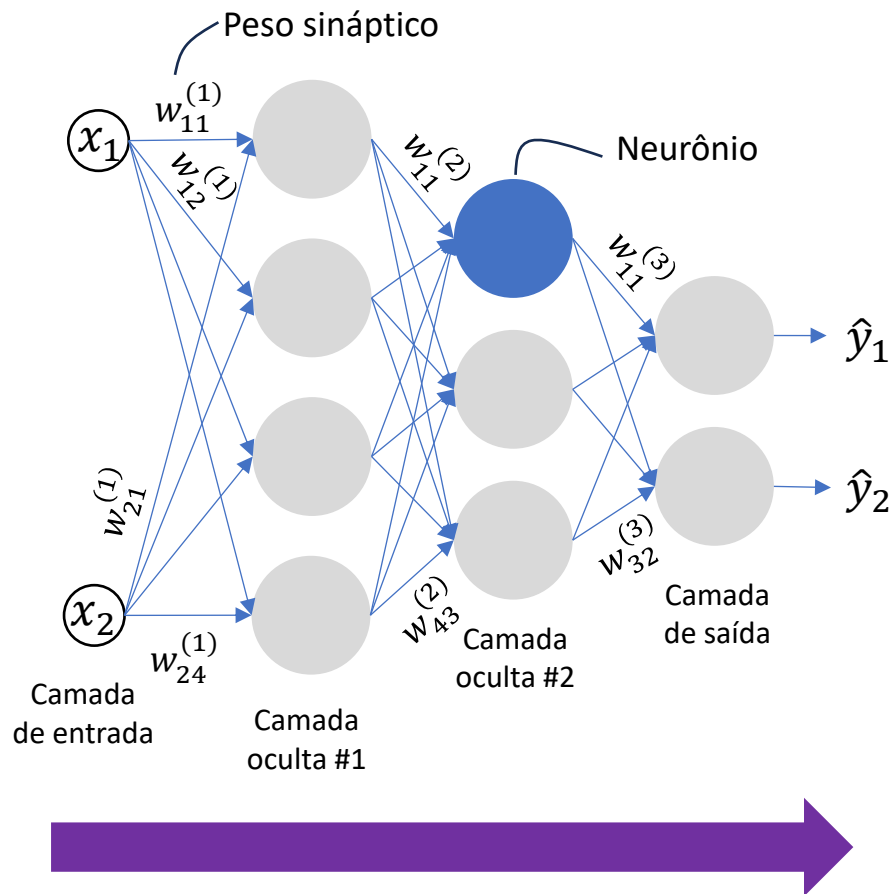
MLPs são redes
densamente
conectadas e de
alimentação direta

Redes neurais artificiais



- As MLPs também são chamadas de **redes densamente conectadas e de alimentação direta** (do inglês, *Dense Neural Networks* - DNNs).
 - Cada uma das saídas de uma camada **se conecta a todos os nós** da camada seguinte através de pesos sinápticos.
 - Os **dados fluem através da rede em uma única direção**, da camada de entrada para a camada de saída, **sem realimentação**.

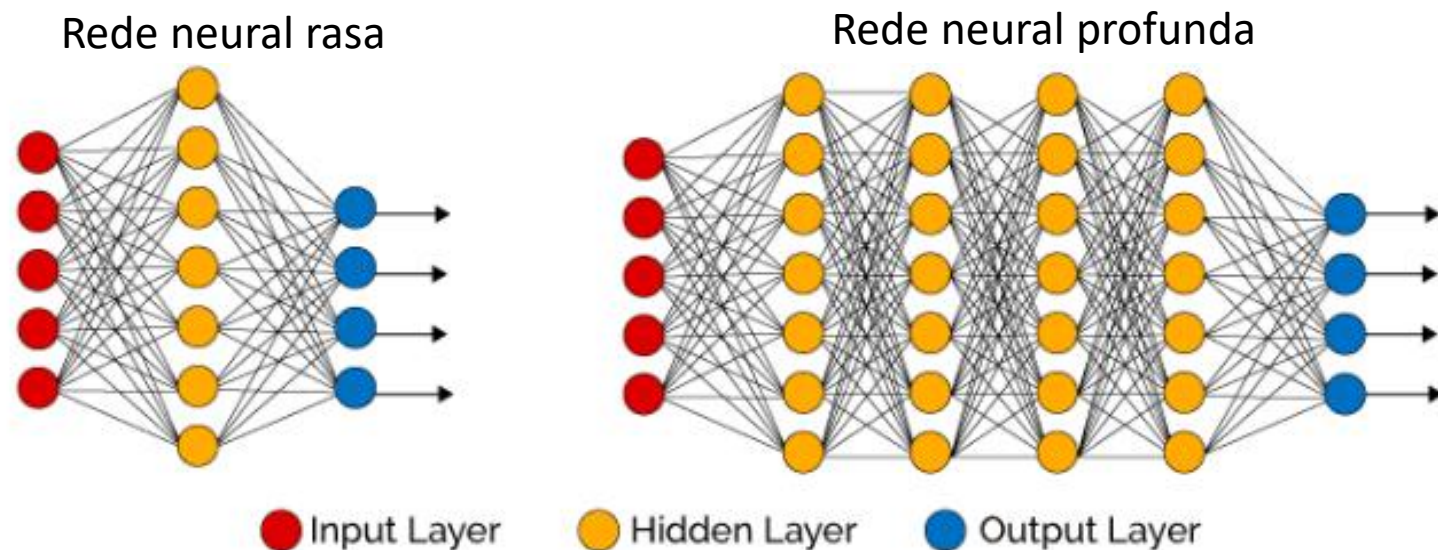
Redes neurais artificiais



- As **propriedades de uma rede neural** são determinadas por sua **arquitetura**.
 - Ela define o que a rede é **capaz de aprender** e **quão bem ela fará isso**.
- A arquitetura é definida pela
 - forma como os neurônios estão conectados (forma direta ou recursiva),
 - quantidade neurônios por camada,
 - quantidade de camadas escondidas,
 - funções de ativação,
 - etc.

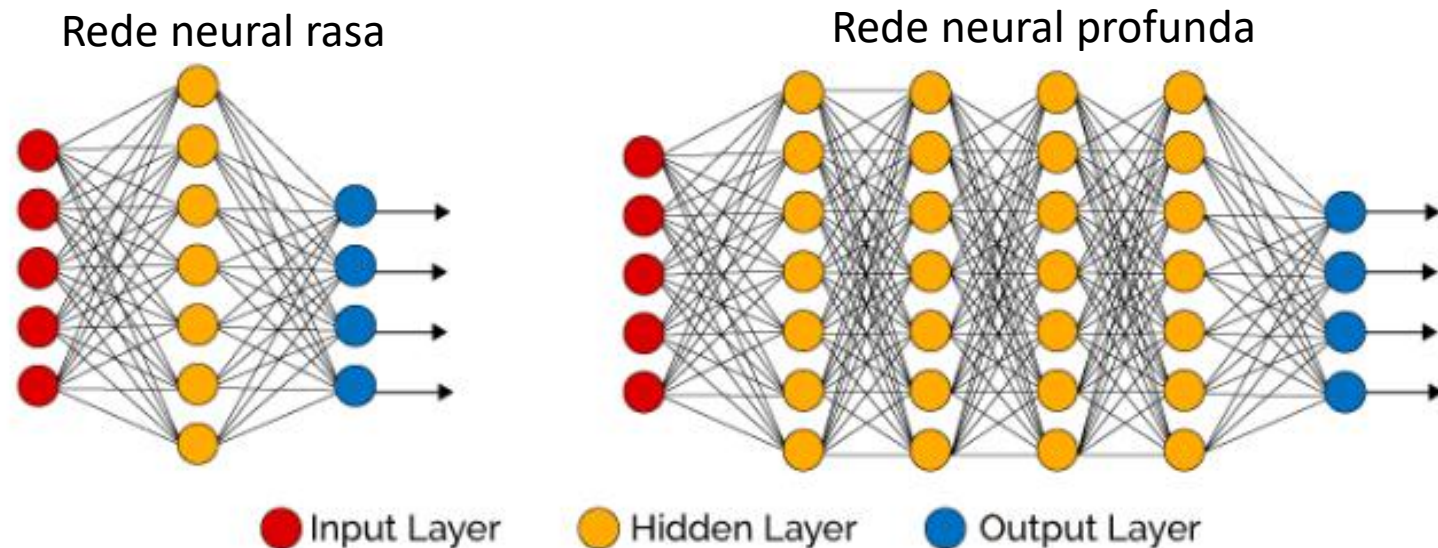
Redes neurais artificiais

- As RNAs são o coração do ***deep learning*** ou **aprendizado profundo**.
- O termo "**profundo**" vem fato destas redes **poderem possuir muitas camadas ocultas**.
- Em geral, quanto mais profunda, maior é a **capacidade de aprendizado** de uma rede neural, porém, maior será sua **complexidade computacional**.



Redes neurais artificiais

- Geralmente, quando elas têm **duas ou mais camadas ocultas**, elas podem ser chamadas de **redes neurais profundas** (*Deep Neural Networks* - DNNs).
- Por poderem ter uma grande capacidade de aprendizado, elas precisam de uma **grande quantidade de dados para aprender uma solução geral e não se sobreajustarem**.



Aproximação de funções

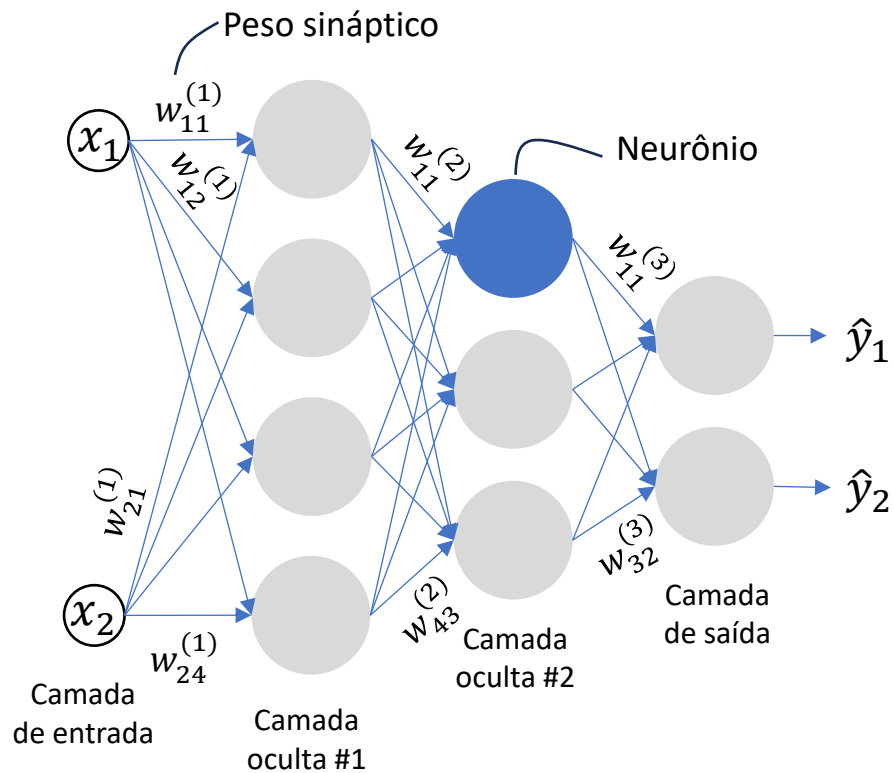
- As redes neurais têm a **capacidade de aproximar funções altamente não-lineares (e lineares também)**.
- Essa capacidade **depende da sua arquitetura**.
 - A **quantidade de pesos** de uma rede está associada aos seus **graus de liberdade**, ou seja, à **capacidade da rede de aproximar diferentes tipos de funções**.
 - Quanto maior a rede, mais pesos e, conseqüentemente, maior é sua capacidade de aprendizado.
- Portanto, assim como polinômios, que podem **aproximar qualquer tipo de função** (linear ou não linear) devido a seus **graus de liberdade**, as **redes neurais podem fazer o mesmo**, bastando apenas que **encontremos sua complexidade ideal**, ou seja, sua **arquitetura ideal**.

Aproximação universal de funções

- Por exemplo, uma rede neural com **uma camada oculta** com um número **suficiente de nós** pode **aproximar** praticamente qualquer **função contínua**.
- Com **duas camadas ocultas**, até **funções descontínuas** podem ser **aproximadas**.
- Portanto, dizemos que as redes neurais possuem **capacidade de aproximação universal** de funções.
- Desta forma, as redes neurais podem resolver **problemas de regressão e de classificação** e uma grande gama de outros problemas.
- O desafio é encontrar a arquitetura e os pesos ideais para a aproximação.
 - Podemos usar **validação cruzada** e uma versão do **gradiente descendente**.

Como as redes neurais
aprendem?

Backpropagation

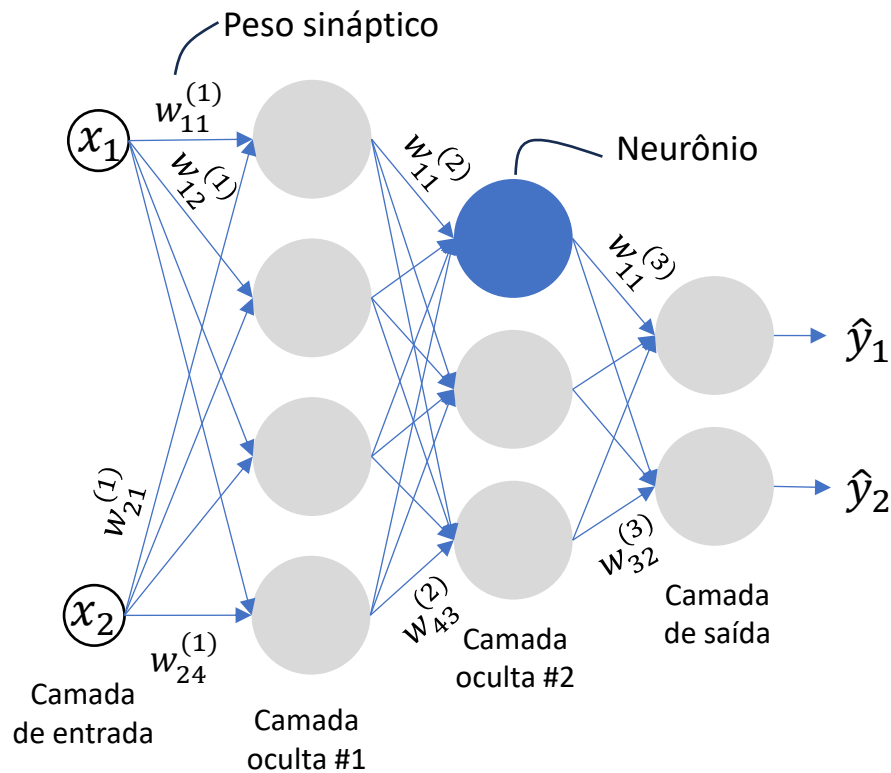


- As redes aprendem ajustando os pesos sinápticos e de *bias* para reduzir o erro.
- Isso é feito através de um algoritmo, baseado no gradiente descendente, chamado de *(error) backpropagation*.
- O **objetivo** do *backpropagation* é calcular o **quanto cada peso da rede contribui para o erro** e **ajustá-lo de forma a minimizar o erro**.



O algoritmo de *error backpropagation* é baseado na **regra da cadeia**.

Backpropagation



- O algoritmo faz três coisas principais:
 - Apresenta os exemplos à rede e ela faz previsões com os pesos atuais.
 - Etapa chamada de direta (*forward*).
 - Calcula o erro entre a saída da rede e os rótulos.
 - **Propaga o erro para trás** através das camadas da rede para atualizar os pesos usando o gradiente descendente.
 - Etapa chamada de retropropagação do erro (*error backpropagation*).
 - As **derivadas parciais do erro em relação aos pesos (gradientes)** são calculadas usando-se a **regra da cadeia**.

Backpropagation

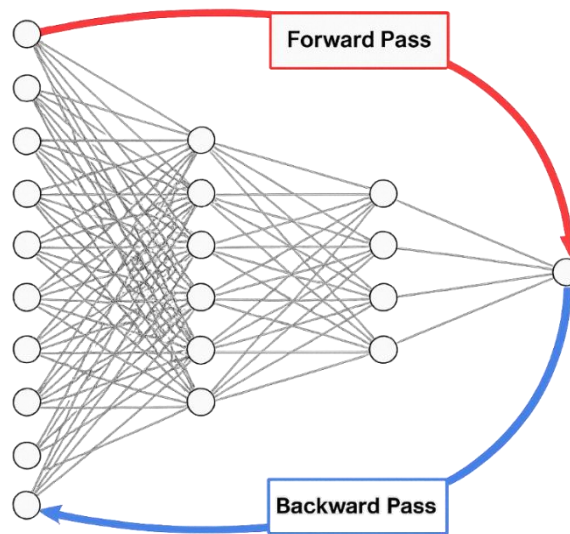
- **Em resumo:** a rede prediz → mede o erro → corrige os pesos.

forward

backward

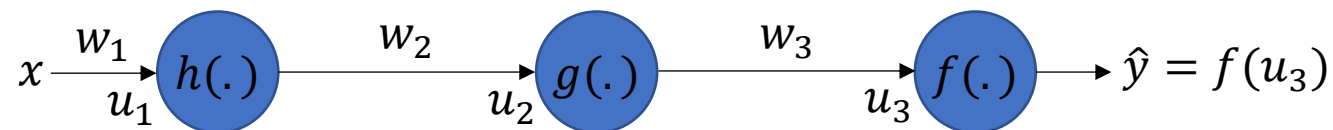
- O *backpropagation* responde à pergunta:

“O quanto os pesos precisam mudar para que a rede produza uma saída mais correta a cada iteração?”



Regra da cadeia

- Para **atualizar o peso de um nó de uma camada qualquer**, a **retropropagação** calcula a derivada parcial do erro em relação àquele peso usando a **regra da cadeia**.
- Vejamos o exemplo abaixo com 3 nós, 3 pesos, w_1 , w_2 e w_3 , 3 funções de ativação, $f(\cdot)$, $g(\cdot)$ e $h(\cdot)$, uma entrada x , uma saída $\hat{y}$ e um rótulo y .



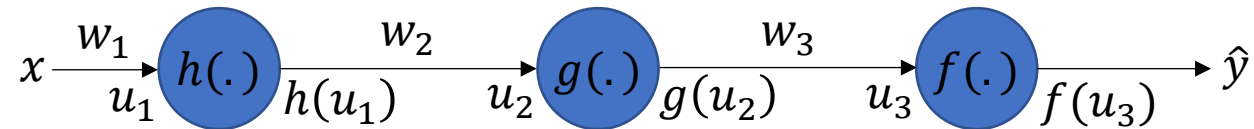
Rede com 3 camadas.

- Para facilitar a análise, vamos definir as entradas dos nós como

$$\begin{aligned}u_1 &= w_1 x, \\u_2 &= w_2 h(w_1 x) = w_2 h(u_1), \\u_3 &= w_3 g(w_2 h(w_1 x)) = w_3 g(u_2).\end{aligned}$$

- Essas variáveis são chamadas de **ativações**.

Regra da cadeia



- Qual é a derivada do erro $J_e = (y - \hat{y})^{2*}$ em relação à w_1 ?

$$\frac{\partial J_e}{\partial w_1} = \frac{\partial (y - f(u_3))^2}{\partial f(u_3)} \frac{\partial f(u_3)}{\partial u_3} \frac{\partial u_3}{\partial g(u_2)} \frac{\partial g(u_2)}{\partial u_2} \frac{\partial u_2}{\partial h(u_1)} \frac{\partial h(u_1)}{\partial u_1} \frac{\partial u_1}{\partial w_1}.$$

- Calculando as derivadas, exceto as das funções de ativação, temos

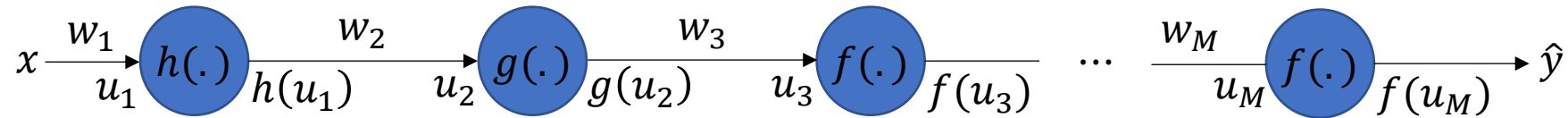
$$\frac{\partial J_e}{\partial w_1} = -2 \left(y - \underbrace{f(u_3)}_{\hat{y}} \right) \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x.$$

- Exemplos:** derivadas das funções de ativação logística e tanh

- Logística: $f(x)(1 - f(x)) \in [0, 0.25]$
- Tanh: $1 - \tanh^2(x) \in [0, 1]$ (**OBS.:** 1 apenas quando $x = 0$)

* Vamos considerar o erro de apenas uma amostra.

Dissipação e explosão do gradiente



- Imaginem uma rede profunda com M camadas. A derivada com relação a w_1 é

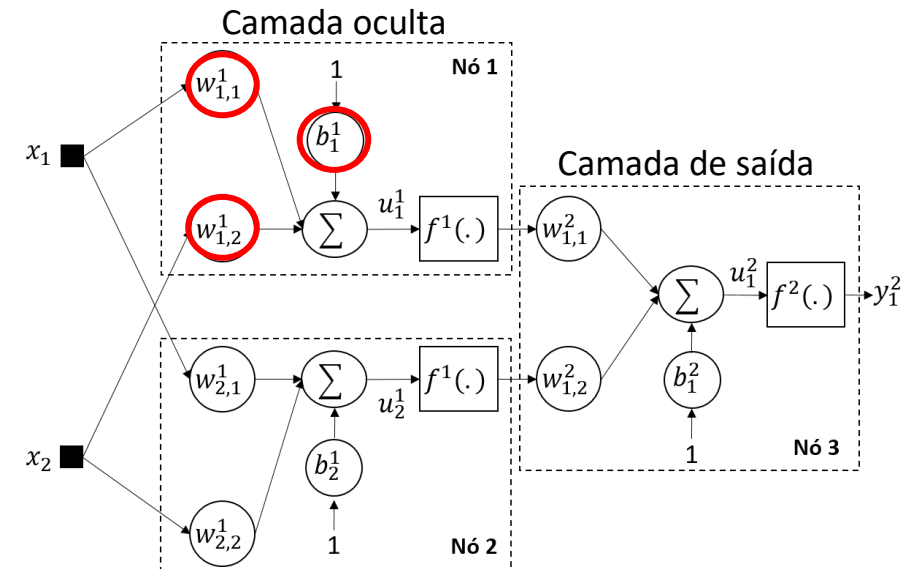
$$\frac{\partial J_e}{\partial w_1} = -2 \left(y - \underbrace{f(u_M)}_{\hat{y}} \right) \frac{\partial f(u_M)}{\partial u_M} w_M \cdots \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x.$$

- Se as derivadas das funções de ativação têm valores < 1 , teremos a multiplicação de M valores < 1 , causando a dissipação do gradiente.
 - Camadas iniciais não aprendem ou aprendem muito lentamente.
- Se os pesos, que são inicializados aleatoriamente, tiverem valores > 1 , teremos a multiplicação de M valores > 1 , causando a explosão do gradiente.
 - Estouro da precisão numérica, instabilidade durante o treinamento e divergência.

Exemplo da aplicação da retropropagação

- Encontrar o **vetor gradiente** para todos os pesos do nó 1 da camada oculta da rede neural MLP abaixo.

$$\begin{bmatrix} \frac{\partial J_e}{\partial w_{1,1}^1} \\ \frac{\partial J_e}{\partial w_{1,2}^1} \\ \frac{\partial J_e}{\partial b_1^1} \end{bmatrix} = ?$$



- **OBS.:** vamos deixar as derivadas das funções de ativação em relação às ativações de forma genérica, i.e., não vamos assumir nenhum tipo específico de função de ativação.

Exemplo da aplicação da retropropagação

- Usaremos o **erro quadrático médio** como função de erro

$$\begin{aligned} J_e &= \frac{1}{N_{\text{dados}} N_M} \sum_{n=1}^{N_{\text{dados}}} \sum_{k=1}^{N_M} e_k^2(n) \\ &= \frac{1}{N_{\text{dados}} N_M} \sum_{n=1}^{N_{\text{dados}}} \sum_{k=1}^{N_M} \left(y_k(n) - \hat{y}_k^M(n) \right)^2, \end{aligned}$$

onde N_{dados} é o número de amostras, N_M é o número de nós na camada de saída, $y_k(n)$ e $\hat{y}_k^M(n)$ são o rótulo da k -ésima saída e a saída do k -ésimo nó da camada de saída, respectivamente, ambos correspondentes à n -ésima amostra de entrada, $\mathbf{x}(n)$ e M representa o índice da última camada.

Exemplo da aplicação da retropropagação

- A rede do exemplo possui **uma camada oculta** com **dois nós** e **uma camada de saída** com **um único nó**, i.e., $N_M = 1$ e $M = 2$.
- Além disso, por motivos didáticos, vamos considerar um único exemplo de entrada, $\mathbf{x} = [x_1, x_2]$ e seu respectivo rótulo, y .

- Assim, $N_{\text{dados}} = 1$, fazendo com que o erro se torne

$$J_e = e_1^2 = (y - \hat{y}_1^2)^2,$$

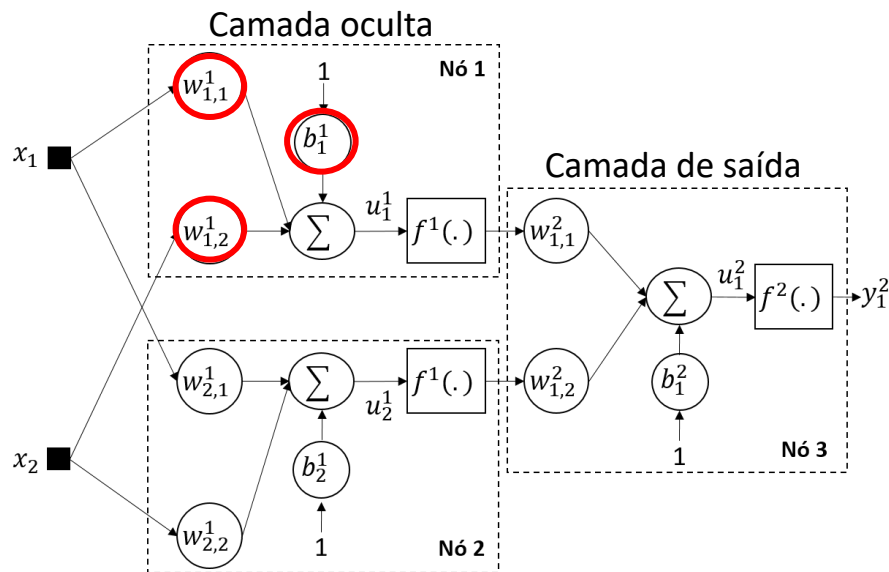
onde o índice n não é apresentado, pois temos apenas uma amostra.

- Assim, o gradiente é dado por

$$\frac{\partial J_e}{\partial w_{i,j}^m} = \frac{\partial e_1^2}{\partial w_{i,j}^m}.$$

OBS.: $\hat{y}_1^2$ não significa “ao quadrado”, mas sim a indicação do índice da camada.

Exemplo da aplicação da retropropagação



- Vamos supor que os pesos de todos os nós são **inicializados de forma aleatória**.
- Assim, quando a entrada, $\mathbf{x}$, é apresentada à rede, é possível calcular todos os valores de interesse ao longo dela até sua saída.
- Consequentemente, tendo o valor de saída, conseguimos calcular o erro.
- Essa é a etapa **direta (*forward*)**.
- Agora, na etapa reversa, calculamos a derivada do erro em relação aos pesos.

Exemplo da aplicação da retropropagação

- Aplicando a regra da cadeia, a derivada parcial do erro em relação ao peso $w_{1,1}^1$ é dada por

$$\frac{\partial J_e}{\partial w_{1,1}^1} = \frac{\partial \left(y - f^2(u_1^2) \right)^2}{\partial f^2(u_1^2)} \frac{\partial f^2(u_1^2)}{\partial u_1^2} \frac{\partial u_1^2}{\partial f^1(u_1^1)} \frac{\partial f^1(u_1^1)}{\partial u_1^1} \frac{\partial u_1^1}{\partial w_{1,1}^1}.$$

- Resolvendo as derivadas parciais, temos

$$\frac{\partial J_e}{\partial w_{1,1}^1} = -2 \left(y - \underbrace{f^2(u_1^2)}_{\hat{y}_1^2} \right) f'^2(u_1^2) w_{1,1}^2 f'^1(u_1^1) x_1,$$

onde $f'^m(u_i^m) = \frac{\partial f^m(u_i^m)}{\partial u_i^m}$ é a derivada parcial da função de ativação do i -ésimo nó da m -ésima camada em relação a sua ativação.

Exemplo da aplicação da retropropagação

- Aplicando-se o mesmo procedimento aos outros pesos, obtemos

$$\frac{\partial J_e}{\partial w_{1,1}^1} = \frac{\partial e^2}{\partial w_{1,1}^1} = \frac{\partial \left(y - f^2(u_1^2) \right)^2}{\partial f^2(u_1^2)} \frac{\partial f^2(u_1^2)}{\partial u_1^2} \frac{\partial u_1^2}{\partial f^1(u_1^1)} \frac{\partial f^1(u_1^1)}{\partial u_1^1} \frac{\partial u_1^1}{\partial w_{1,1}^1}$$

$$\frac{\partial J_e}{\partial w_{1,2}^1} = \frac{\partial e^2}{\partial w_{1,2}^1} = \frac{\partial \left(y - f^2(u_1^2) \right)^2}{\partial f^2(u_1^2)} \frac{\partial f^2(u_1^2)}{\partial u_1^2} \frac{\partial u_1^2}{\partial f^1(u_1^1)} \frac{\partial f^1(u_1^1)}{\partial u_1^1} \frac{\partial u_1^1}{\partial w_{1,2}^1}$$

$$\frac{\partial J_e}{\partial b_1^1} = \frac{\partial e^2}{\partial b_1^1} = \frac{\partial \left(y - f^2(u_1^2) \right)^2}{\partial f^2(u_1^2)} \frac{\partial f^2(u_1^2)}{\partial u_1^2} \frac{\partial u_1^2}{\partial f^1(u_1^1)} \frac{\partial f^1(u_1^1)}{\partial u_1^1} \frac{\partial u_1^1}{\partial b_1^1}$$

- Percebam que todos os termos são iguais, exceto o último, cuja derivada resulta na entrada à qual o peso está conectado.

Exemplo da aplicação da retropropagação

- Assim, o vetor gradiente com as derivadas parciais do erro com relação a todos os pesos do **nó** $i = 1$ da camada $m = 1$ é apresentado abaixo

$$\begin{bmatrix} \frac{\partial J_e}{\partial w_{1,1}^1} \\ \frac{\partial J_e}{\partial w_{1,2}^1} \\ \frac{\partial J_e}{\partial b_1^1} \end{bmatrix} = \underbrace{-2(y - \hat{y}_1^2)f'^2(u_1^2)w_{1,1}^2f'^1(u_1^1)}_{\text{escalar}} \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix}.$$

Os pesos de **bias** estão ligados às entradas de todos os nós com valores constantes iguais a 1.

Pseudocódigo da retropropagação

- Inicialize a matriz de pesos, $\mathbf{W}$ (sinápticos e de *bias*) e a taxa de aprendizagem, α .
- Faça $k = 0$ (contador de épocas)
- Calcule o erro inicial $J(\mathbf{W}(k))$ (opcional)
- Enquanto o critério de parada não for atendido, faça:
 - Para l variando de 1 até L , faça:
 - Retire uma amostra aleatoriamente sem reposição do conjunto, a apresente à rede e calcule o erro.
 - Calcule $\nabla J_l(\mathbf{W}(k))$
 - $\mathbf{W}(k + 1) = \mathbf{W}(k) - \frac{\alpha}{L} \sum_{l=1}^L \nabla J_l(\mathbf{W}(k))$
 - Calcule o erro da k -ésima iteração $J(\mathbf{W}(k))$ (opcional)
 - $k = k + 1$

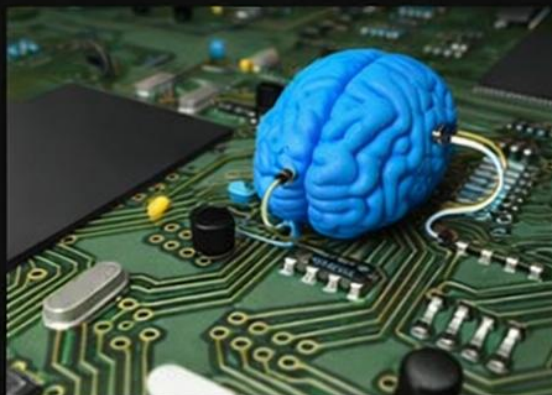
onde L é o número de amostras usadas para estimar o gradiente, $\nabla J_l(\mathbf{W}(k)) = \frac{1}{N_M} \sum_{j=1}^{N_M} \frac{\partial e_j^2(l)}{\partial \mathbf{W}(k)}$ é a média da derivada parcial do erro para todas as N_M saídas, mas considerando apenas a l -ésima amostra.

OBS.: O pseudocódigo apresentado **implementa qualquer versão do gradiente descendente** variando-se o valor de L .

Vou ter que implementar tudo
isso manualmente?



What society thinks I do



What my friends think I do



What other computer
scientists think I do



What mathematicians think I do



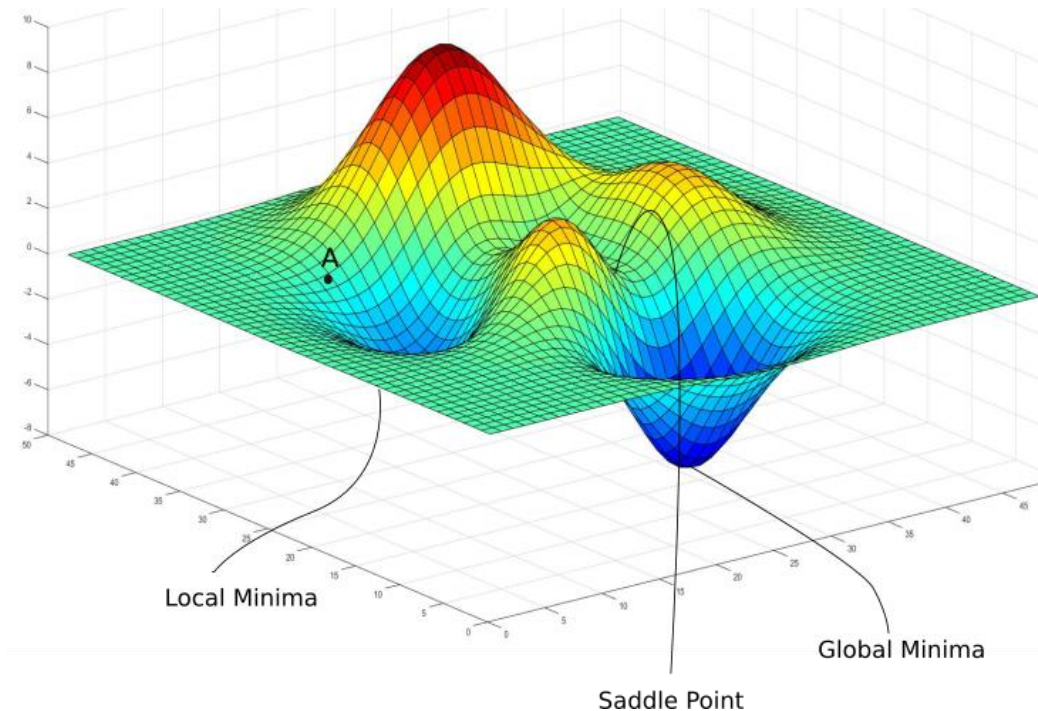
What I think I do

```
from tensorflow import *
```

What I actually do

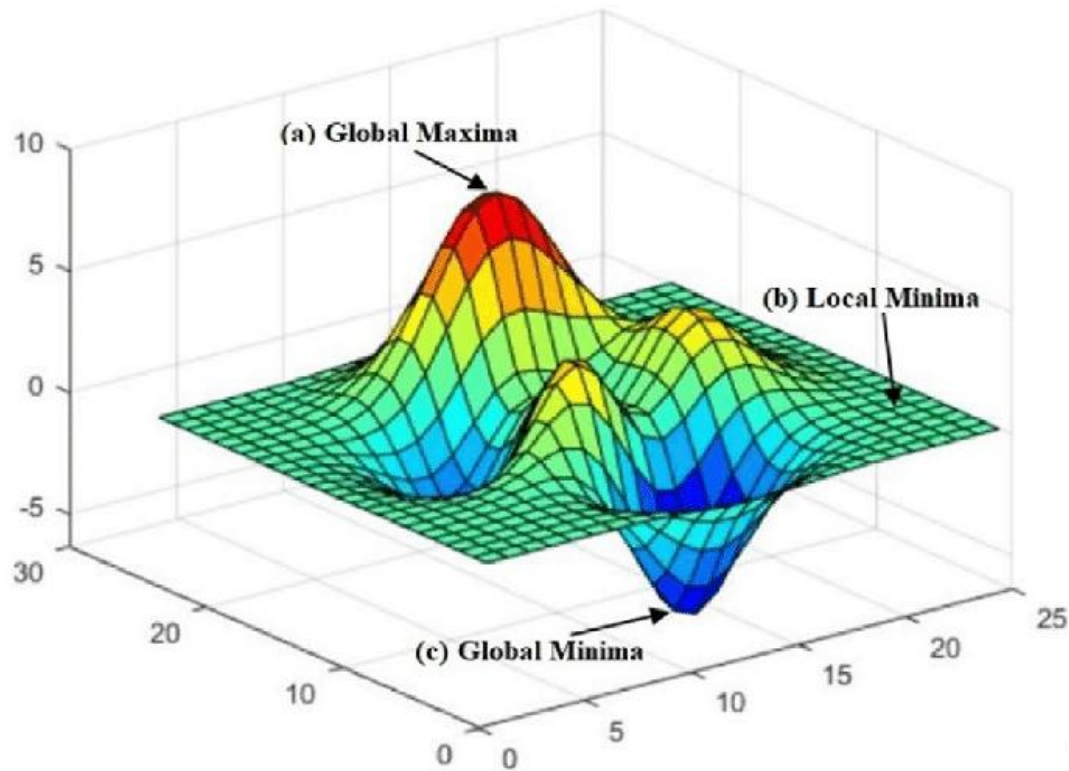
Não se preocupem, hoje em dia, não realizamos esse procedimento de forma manual, **usamos bibliotecas que automatizam o processo.**

Superfícies de erro de redes neurais



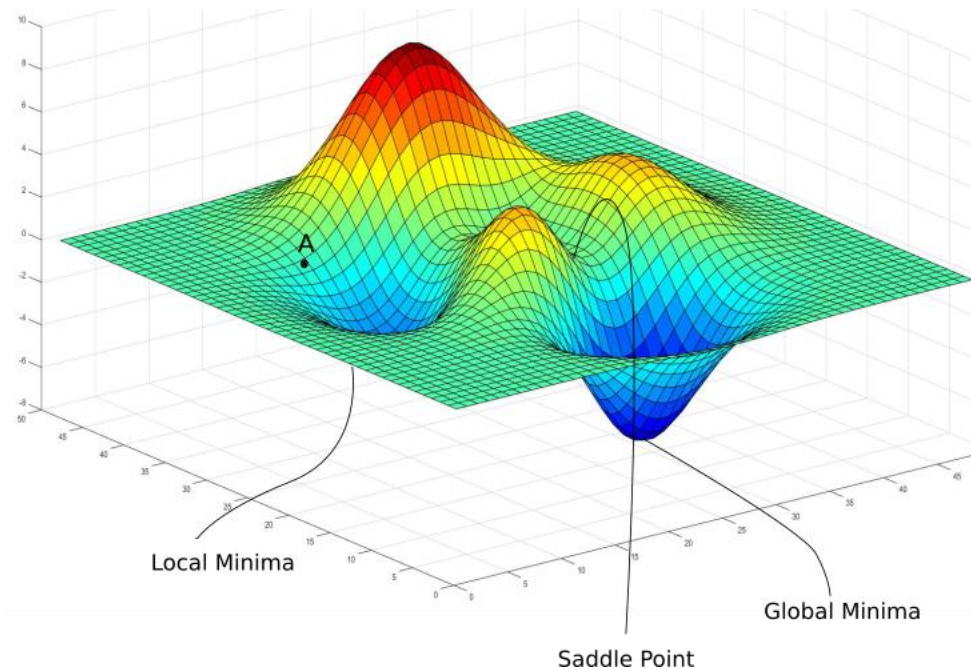
- Por serem formadas pela **combinação de vários nós com funções de ativação não-lineares**, as superfícies de erro de redes neurais **não são convexas**.
- Elas são **altamente irregulares**, incluindo irregularidades como
 - Pontos de sela: o erro diminui em algumas direções e aumenta em outras.
 - Vários mínimos locais.
 - Platôs: região plana com erro alto.
 - etc.

Superfícies de erro irregulares



- Essas irregularidades causam
 - treinamento mais lento
 - instabilidade numérica
 - soluções diferentes para o mesmo problema
- Ou seja, as **irregularidades dificultam o aprendizado.**

Superfícies de erro irregulares



- Felizmente, como mostrado em [1, 2], em geral, **quase todos os mínimos locais têm erro próximo ao do mínimo global** e, portanto, encontrar um desses mínimos locais já é bom o suficiente para um dado problema.
- Isso explica porque o **gradiente descendente funciona bem mesmo em problemas altamente não convexos**.

[1] Anna Choromanska, Yann LeCun et al., "The Loss Surfaces of Multilayer Networks", 2015

[2] Benjamin D. Haeffele, Rene Vidal, "Global Optimality in Neural Network Training", 2017

Mas como garantir que o valor de erro encontrado é bom o suficiente?

O erro encontrado é suficientemente bom?

- Devemos **avaliar a generalização** em dados não utilizados no treinamento.
- O modelo é considerado adequado se apresenta **bom desempenho fora do conjunto de treinamento**.
 - Ou seja, bom desempenho nos conjuntos de validação e teste.
- Um bom mínimo local, que generalize bem, costuma bastar.

Como avaliar a estabilidade da otimização?

- **Repetir o treinamento** com diferentes inicializações aleatórias (i.e., pesos iniciais).
- **Comparar as métricas** de treino, validação e teste entre as repetições.
 - Reportar **média e desvio padrão** (por exemplo, 5 a 10 execuções).
- **Desempenho semelhante**: evidência de estabilidade da otimização.
 - O algoritmo encontra, **de forma consistente**, soluções boas.
- **Desempenho muito diferente**: investigar possíveis causas:
 - Taxa de aprendizado inadequada
 - Sensibilidade à inicialização
 - Treinamento insuficiente (poucas épocas)
 - Mínimos de qualidade muito distinta (algoritmo converge para pontos diferentes)
- **Cuidado**: estabilidade não garante que o mínimo global foi encontrado.

Exemplo

- [Exemplo: mlp.ipynb](#)



Lista de exercícios

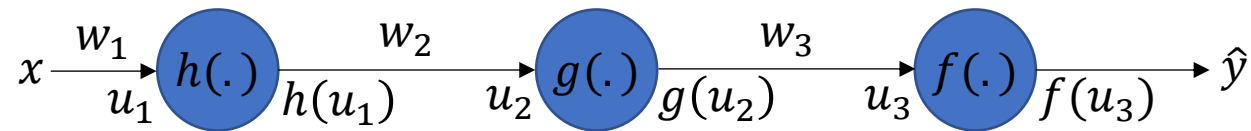
[Lista #9 - MLP](#)

Perguntas?

Obrigado!

Anexo I: Desaparecimento e explosão do gradiente

Derivada parcial do erro



- Vamos usar novamente a rede com **3 camadas** acima como exemplo.
- Como já vimos, a derivada parcial do erro em relação ao peso w_1 do nó da primeira camada é dada por

$$\frac{\partial J_e}{\partial w_1} = \frac{\partial (y - f(u_3))^2}{\partial f(u_3)} \frac{\partial f(u_3)}{\partial u_3} \frac{\partial u_3}{\partial g(u_2)} \frac{\partial g(u_2)}{\partial u_2} \frac{\partial u_2}{\partial h(u_1)} \frac{\partial h(u_1)}{\partial u_1} \frac{\partial u_1}{\partial w_1}.$$

- Podemos reescrevê-la deixando apenas as derivadas das funções de ativação de forma genérica

$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

Derivada parcial do erro

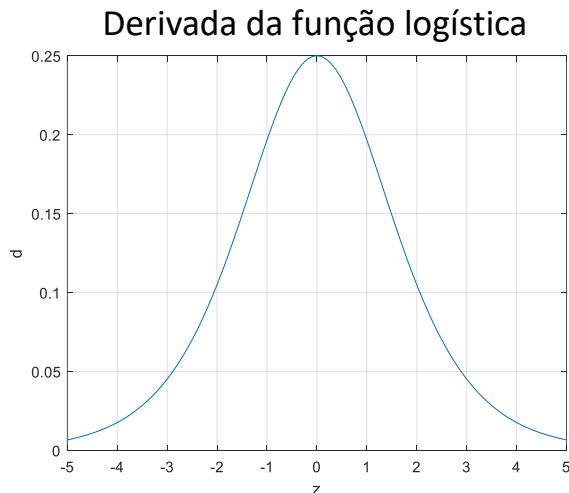
$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

- A rede com 3 camadas tem
 - 3 derivadas parciais das funções de ativação
 - e 2 pesos que ligam a saída à camada de entradapara a atualização do peso do nó da primeira camada.
- Se extrapolarmos a equação acima para redes com M camadas, teremos a multiplicação de
 - M derivadas parciais das funções de ativação
 - e $M - 1$ pesospara a derivada parcial do erro em relação ao peso da primeira camada.

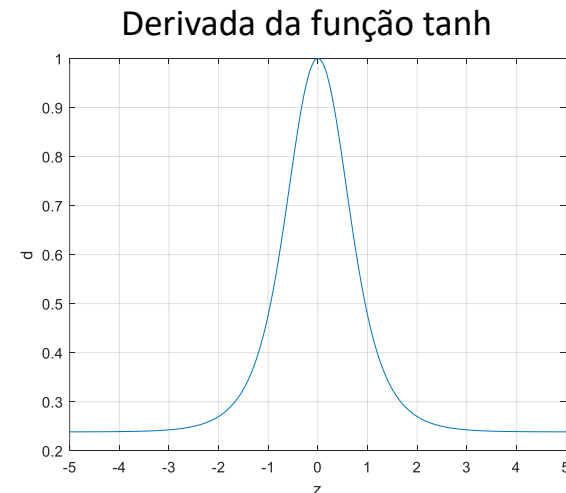
Desaparecimento do gradiente

$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

- As derivadas das funções logística e tangente hiperbólica têm valores máximos iguais a 0.25 e 1, respectivamente, apenas quando as ativações são iguais a 0.



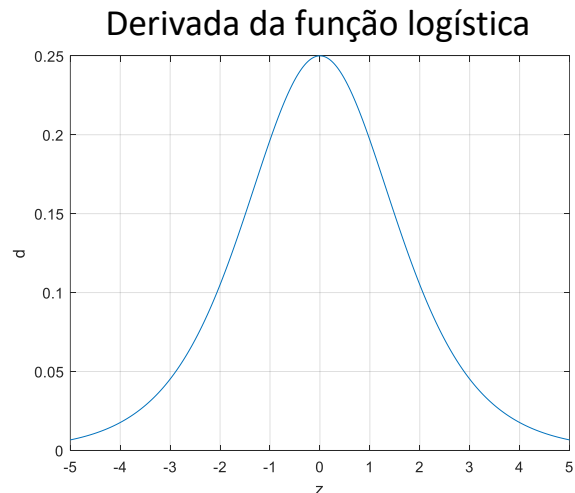
$$f(z)(1 - f(z)) \in [0, 0.25]$$



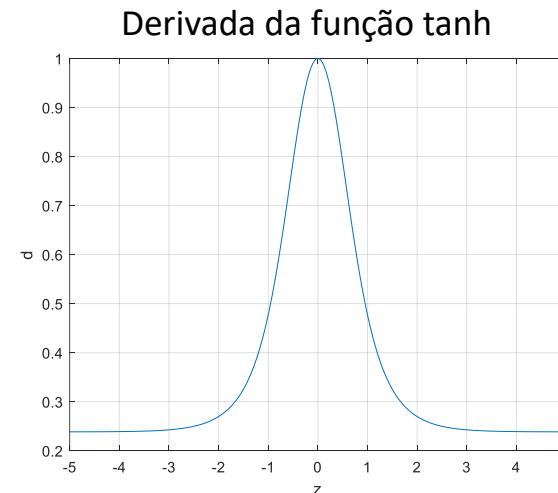
$$1 - \tanh^2(z) \in [0, 1]$$

Desaparecimento do gradiente

- Se usarmos a função logística, teremos a multiplicação de até M termos menores do que 0.
 - No máximo iguais a 0.25 quando a entrada é igual a 0.
- Usando a função tanh, teremos a multiplicação de até M termos com valores no máximo iguais a 1, mas apenas quando a entrada é igual a 0.



$$f(z)(1 - f(z)) \in [0, 0.25]$$



$$1 - \tanh^2(z) \in [0, 1]$$

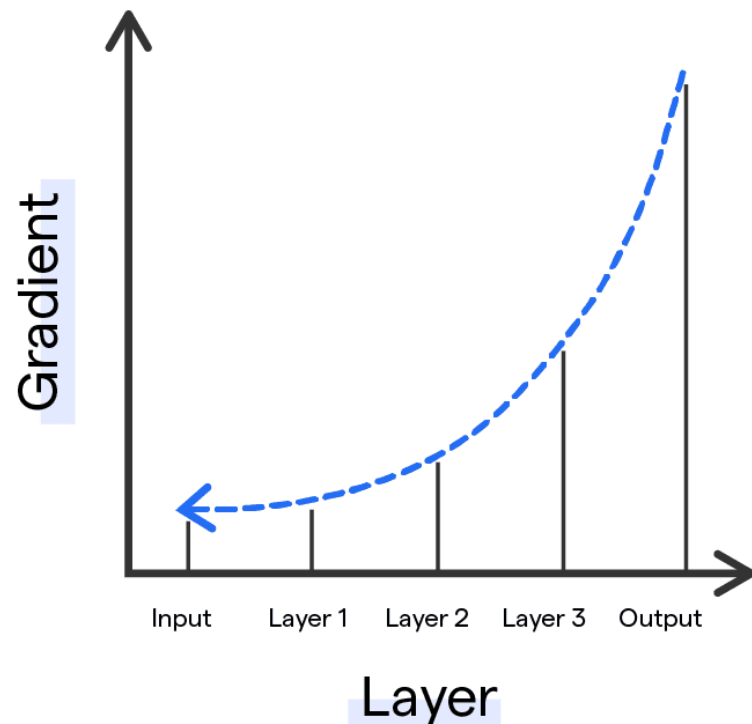
Desaparecimento do gradiente

- Assim, em uma rede com M camadas, a **retropropagação** tem o efeito de multiplicar até M valores, em geral, menores do que 1 para calcular os vetores gradiente das primeiras camadas.
- **OBS.:** Se os valores iniciais dos pesos forem menores do que 1, também teremos a multiplicação de vários termos pequenos.

Mas o que isso significa na prática?

Desaparecimento do gradiente

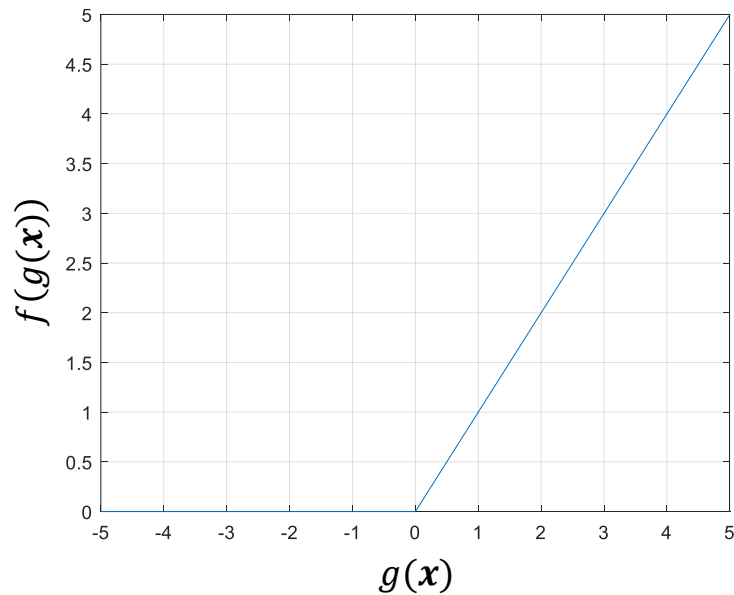
Vanishing Gradient Problem



- Isso significa que o **gradiente diminui exponencialmente com M** .
- Assim, os **nós das camadas iniciais aprendem muito mais lentamente** do que os nós das camadas finais, pois o **vetor gradiente** daquelas camadas é **muito pequeno**, fazendo com que a **atualização dos pesos também seja pequena**.
- Isso faz com que as **primeiras camadas aprendam lentamente ou nem aprendam**, pois têm **derivadas muito pequenas, tendendo a zero**.

**Como mitigar esse
problema?**

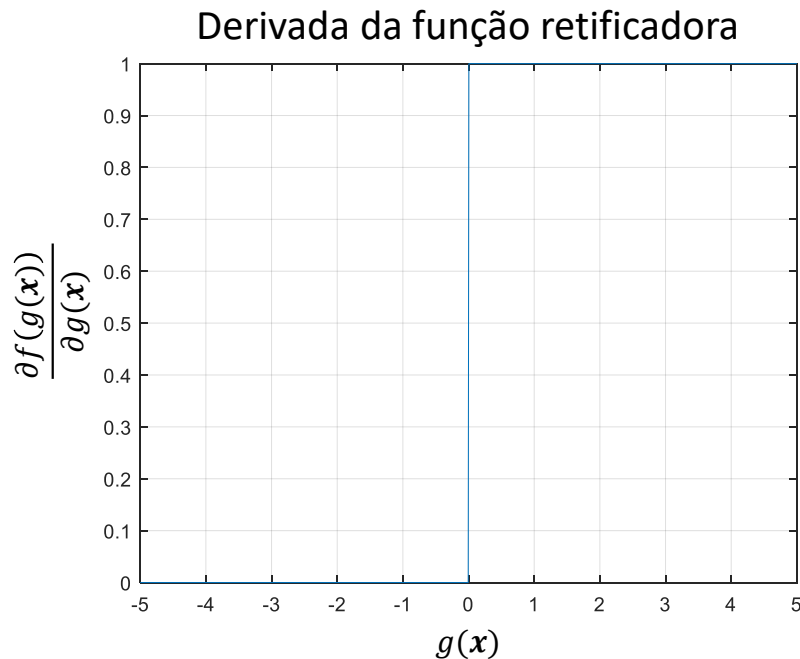
Função de ativação retificadora



$$\hat{y} = f(g(x)) = \max(0, g(x))$$

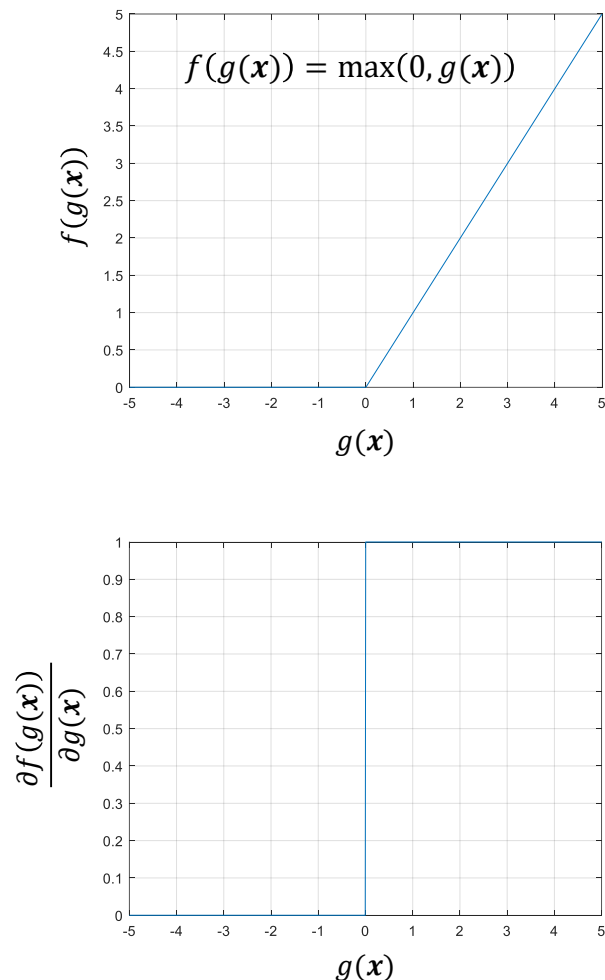
- Com o surgimento das **redes neurais profundas**, e, conseqüentemente, do problema do **desaparecimento do gradiente**, uma outra função de ativação, conhecida como ***Rectified Linear Unit*** (ReLU), passou a ser bastante utilizada.
- É também uma **função não-linear** onde sua saída é igual 0 quando $g(x) \leq 0$ e o próprio $g(x)$ quando $g(x) > 0$.
- É uma das funções mais utilizadas em redes neurais profundas atualmente.

Função de ativação retificadora



- Sua principal **vantagem** é que ela ajuda a **minimizar o problema do desaparecimento do gradiente**, pois sua derivada é igual a 1 para $g(x) > 0$.
- Sua derivada é dada por
$$\frac{dy_j}{dg(x)} = \frac{df(g(x))}{dg(x)} = \begin{cases} 0, & \text{se } g(x) < 0 \\ 1, & \text{se } g(x) > 0 \end{cases}$$
- A derivada é indeterminada para $g(x) = 0$.
- Outra **vantagem** é a sua **eficiência computacional**.
 - Ela e sua derivada **são mais rápidas de se calcular** do que as funções logística e tangente hiperbólica.

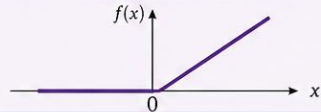
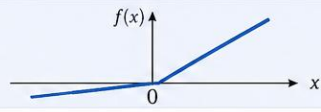
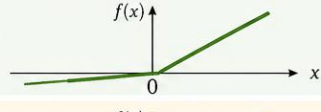
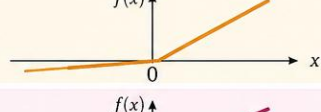
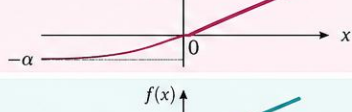
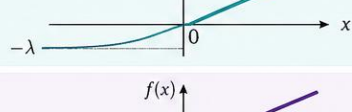
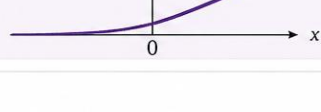
Função de ativação retificadora



- Uma **desvantagem** é que ela causa o problema conhecido como **ReLU agonizante**.
- Esse **problema ocorre durante o treinamento** da rede, quando a ativação do nó, $g(x)$, é **negativa**.
- Isso faz com que sua **saída e**, consequentemente, a **derivada parcial da função de ativação sejam iguais a 0**.
- Quando isso ocorre, o **nó não tem seus pesos atualizados** durante o treinamento, **permanecendo inalterados**.

Variantes da função de ativação retificadora

VARIANTES DA FUNÇÃO DE ATIVAÇÃO ReLU

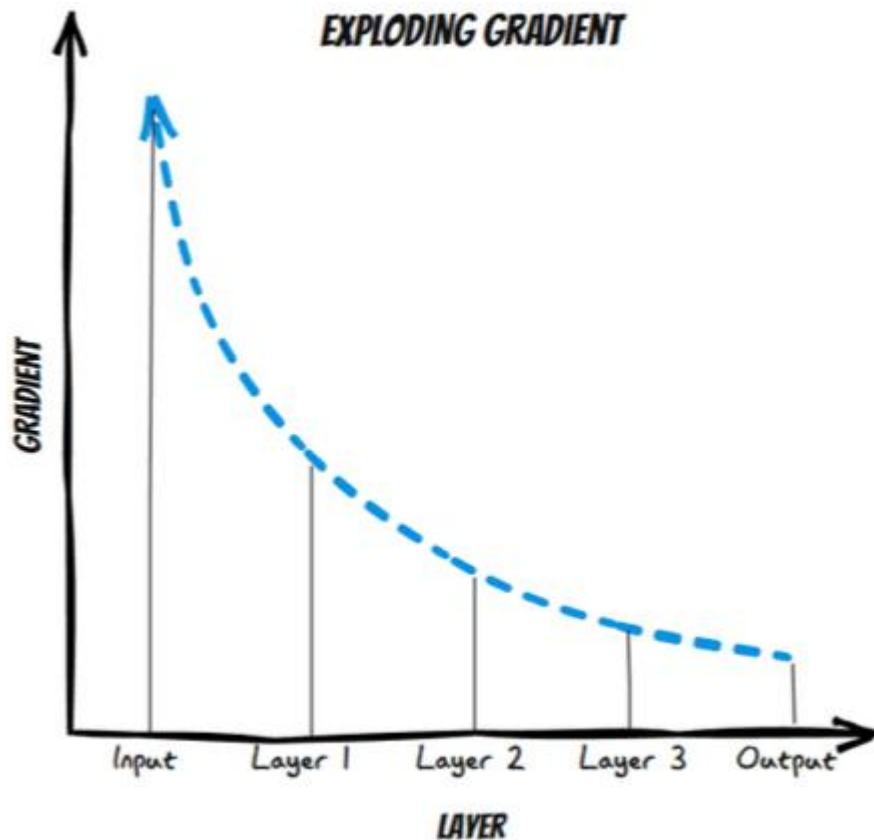
VARIANTE	NOME COMPLETO	FUNÇÃO MATEMÁTICA $f(x)$	GRÁFICO DA FUNÇÃO
ReLU	Rectified Linear Unit	$f(x) = \max(0, x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$	
Leaky ReLU	Leaky Rectified Linear Unit	$f(x) = \begin{cases} \alpha x, & x < 0 \\ x, & x \geq 0 \end{cases} \quad (0 < \alpha \ll 1)$	
PReLU	Parametric ReLU	$f(x) = \begin{cases} ax, & x < 0 \\ x, & x \geq 0 \end{cases} \quad (a \text{ é aprendido})$	
RReLU	Randomized ReLU	$f(x) = \begin{cases} ax, & x < 0 \\ x, & x \geq 0 \end{cases} \quad \begin{matrix} (a \sim \mathcal{U}(l, u) \text{ durante treino;} \\ a = \frac{l+u}{2} \text{ no teste}) \end{matrix}$	
ELU	Exponential Linear Unit	$f(x) = \begin{cases} \alpha(e^x - 1), & x < 0 \\ x, & x \geq 0 \end{cases} \quad (\alpha > 0)$	
SELU	Scaled Exponential Linear Unit	$f(x) = \lambda \begin{cases} e^x - 1, & x < 0 \\ x, & x \geq 0 \end{cases} \quad (\lambda > 0)$	
Softplus	Smooth ReLU	$f(x) = \log(1 + e^x)$	

Onde: $x \in \mathbb{R}$

ReLU e variantes são amplamente utilizadas em redes neurais para introduzir não linearidade.

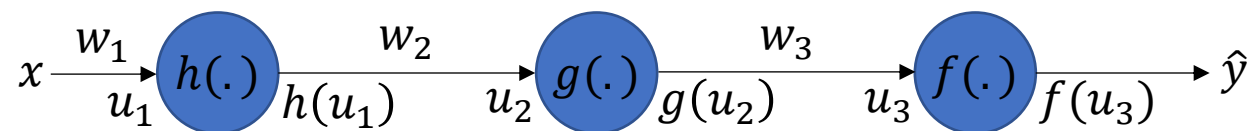
- Para mitigar o problema das **ReLUs agonizantes**, usa-se **variantes da função ReLU** que **possuam derivada diferente de zero para $g(x) < 0$** , como, por exemplo,
 - [Leaky ReLU](#),
 - [Parametric ReLU \(PReLU\)](#),
 - [Gaussian Error Linear Unit \(GELU\)](#),
 - [etc.](#)

Explosão do gradiente



- Usando funções de ativação ReLU e suas variantes, reduzimos o problema do desaparecimento do gradiente.
- Porém, um **outro problema surge** quando as **ativações são positivas (derivadas iguais a 1)** e os **pesos têm valores maiores do que 1**.
- Caso os pesos sejam inicializados (em geral, de forma aleatória) com **valores maiores do que 1**, haverá a **multiplicação de vários valores assim**, o que pode resultar em **valores de gradiente muito grandes nas camadas iniciais**.

Explosão do gradiente



$$\frac{\partial J_e}{\partial w_1} = -2(y - \hat{y})^2 \frac{\partial f(u_3)}{\partial u_3} w_3 \frac{\partial g(u_2)}{\partial u_2} w_2 \frac{\partial h(u_1)}{\partial u_1} x_1.$$

- Se os elementos do vetor gradiente tiverem **magnitudes muito grandes**, os **pesos da rede podem sofrer atualizações extremamente grandes**, o que leva a **instabilidades numéricas** e a um **treinamento ineficaz** ou até mesmo à **divergência**.

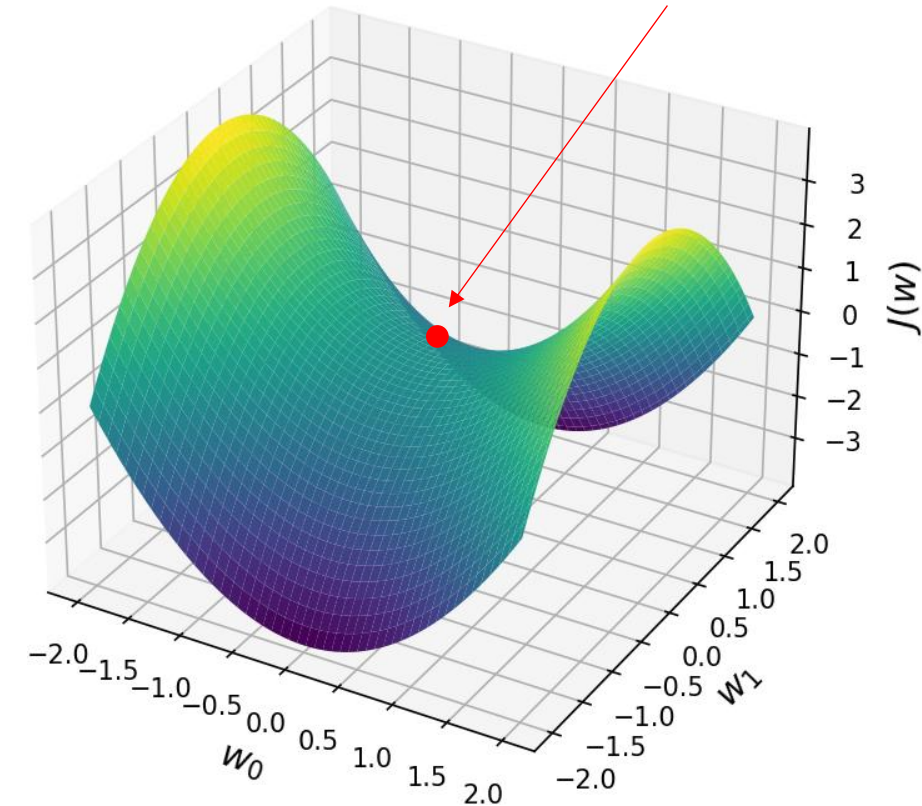
Formas de se minimizar a dissipação e a explosão do gradiente

- Além do uso de funções de ativação ReLU ou de suas variantes, outras formas de se minimizar esses problemas são:
 - **Inicialização apropriada dos pesos:** **garante que a variância das ativações permaneça a mesma** ao longo de todas as camadas. Isso garante que o gradiente retropropagado **não tenha multiplicações com valores muito pequenos ou muito grandes** em qualquer camada, ajudando a mitigar ambos os problemas.
 - **Normalização das ativações:** **padroniza as ativações das camadas** durante o treinamento. Isso estabiliza os gradientes, acelera a convergência e mitiga ambos os problemas.
 - **Poda do gradiente:** **limita o valor máximo do gradiente** durante o treinamento. Isso evita atualizações excessivas dos pesos e previne a explosão do gradiente.
 - **Arquiteturas com conexões residuais:** **criam caminhos diretos entre camadas (curto circuito)**. Isso facilita a propagação do gradiente em redes muito profundas, minimizando ambos os problemas.

Anexo II: Algumas irregularidades que dificultam o aprendizado de redes neurais

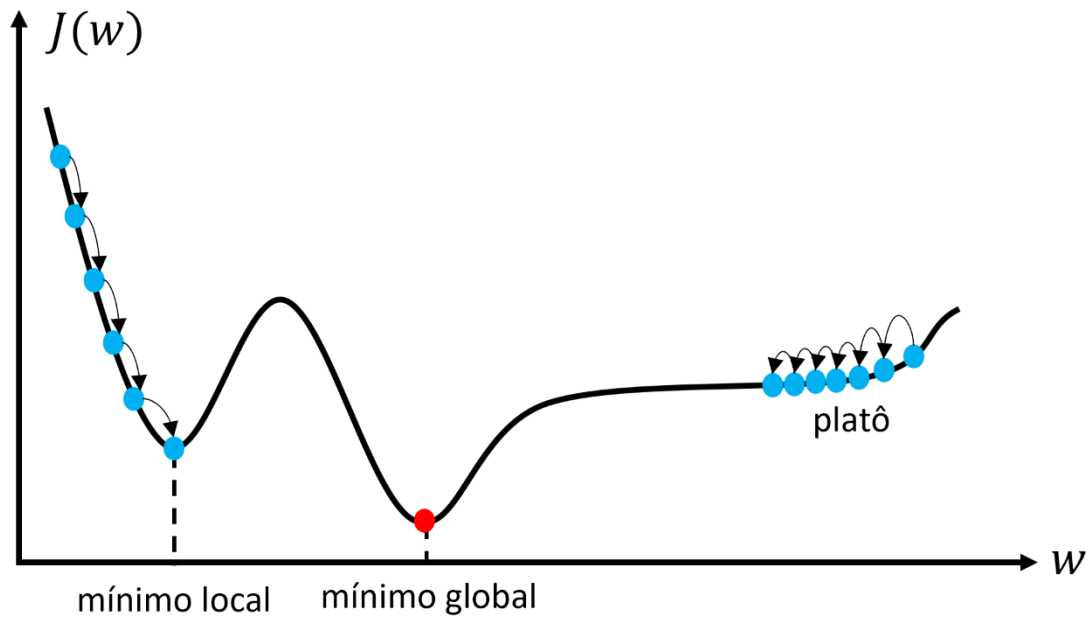
Pontos de sela

Superfície com Ponto de Sela



- Uma irregularidade comumente encontrada em redes neurais são os **pontos de sela**:
 - É um ponto que é um **mínimo ao longo de um eixo**, mas um **máximo ao longo de outro**.
 - Em algumas direções são **atratores** (i.e., alta declividade), mas em outras não.
- O algoritmo de otimização pode passar um longo período de tempo sendo atraído por eles, o que prejudica seu desempenho.
- Para escapar destes pontos, usa-se métodos de **segunda ordem** ou **versões estocásticas** (i.e., ruidosas) do gradiente descendente.

Platôs



- Outro tipo de irregularidade são os **platôs**.
- Eles são **regiões planas e com erro elevado**.
- Como a **inclinação da superfície** nessa região é **próxima de zero** (i.e., o gradiente é próximo de zero) o algoritmo pode levar muito tempo para atravessá-la.
- Métodos de **aprendizado adaptativo**, que são variações do gradiente descendente, como AdaGrad, RMSProp, Adam, podem escapar destas regiões.